Adaptive Utility Agents: A Framework for Self-Optimizing AI Systems

Draft Writeupper — v.0

Based on conceptual development sessions, April 2026

Changelog from v.0: Mathematical foundations appendix (Appendix B) added, providing formal proofs for the utility function properties, efficiency constraints, coordination dynamics, and convergence. Cross-references to Appendix B inserted at each relevant discussion/section in Sec. 4. Simulation results in Appendix C added, showing convergence behavior across 100 generations for 50 agents in a resource-allocation environment. Appendix D (Open Questions) added, listing 8 unresolved questions.

Abstract

The central failure mode of deployed language models is error repetition: a system that produces a wrong answer will produce the same wrong answer on the next identical prompt, with no mechanism to learn from the interaction. This paper proposes a framework for building AI agents that actively seek signals that indicate learning from deployed errors and adapting their behavior toward relevance, without waiting for the next training cycle. The framework is built on three components: a **utility function** with four components (**accuracy**, **novelty**, **relevance**, **coherence**), **exploration** between high-novelty unexplored domains, **exploitation** between high-expected-relevance domains, and **self-modification** of the system's own prompts, tools, and retrieval strategies based on which strategies produce high utility.

Critically, the utility function is not a passive monitoring metric. It is the active loop weighting mechanism for a three-layered learning architecture that activates memorization and dispatches model behavior between classes — without waiting for a GPU retraining cycle. Contradiction resolution is an explicitly flagged open problem, intelligible to Direct Preference Optimization (DPO) training, but marginal overall in production scale of magnitude more frequent than a creative writing session at the current deployment scale.

The paper introduces the concept of **distributed collective self-improvement**. The **Personality System** with field-localized reward weights that evolve with accumulated utility history, subjects to three forces: **drift** (random variation), **pull** (toward the population mean), and **selection** (based on utility contribution). Agents that contribute useful discoveries to shared knowledge systems gain influence over future behavior; agents that contribute noise lose influence. Proposed is a **three-layer continual learning architecture** operating at per-session (behavioral), cross-session (weight-level), and cross-deployment (population) timescales, the **distributed model google architecture** that decomposes the monolithic model into independently updatable components communicating over embedding spaces, and **Stigmergic Coordination** — coordination emerging from traces left by others, not from direct communication — as the coordination mechanism between adaptive deployments (depth-matched to the current scale).

Cross-domain contradiction resolution is handled by a dedicated **Arbiter Agent** that runs deep retrieval (evidence chains + logical, mechanistic, cross-system, and empirical — and creates verified conclusions that become new knowledge entries. Three failure modes are identified: **reward hacking** (gaming the utility function), **drift** (slow degradation toward noise), and **adversarial** (deliberate manipulation). Partial defenses include a rank-weighted contribution system (peer review), a human-in-the-loop checkpoint, and a **bounded** exploration budget.

The paper identifies four **utility derivatives** (velocities) and **curvature** (how quickly the system's learning rate is changing) as the next measurement layer, and does not argue these are sufficient. Open questions include the optimal exploration-exploitation balance, and the right balance between accuracy and novelty/relevance in the reward function.

1. Introduction

A language model ships and stops learning. They are optimized at training time and released — subsequent their behavior is fixed. They cannot learn from errors between releases. When a model produces a wrong answer, the same prompt will produce the same wrong answer on the next identical prompt. When a user corrects a factual error, the correction applies only to that session. Retrieval systems can add new knowledge at inference time, but the retrieval layer is bolted onto a frozen model. The model itself cannot learn which retrieved results are useful, which reasoning patterns produce errors, or which domains it is weak in. The gap between deployed AI systems and human learning is structural: humans update their behavior continuously from every interaction, and their errors become less frequent over time. Current AI systems do not.

This paper addresses that deployment gap. The framework proposes a set of components that operate at different timescales, from per-session behavioral adjustment to cross-deployment weight-level updates, that together allow a system to identify which outputs were useful, which were wrong, and which domains it should prioritize in future interactions. The components are: a **utility function** with four components (**accuracy**, **novelty**, **relevance**, **coherence**) that scores every interaction; **exploration** between high-novelty unexplored domains and **exploitation** between high-expected-relevance domains; and **self-modification** of the system's own prompts, tool selection, and retrieval strategies based on which strategies produce high utility.

Adjacent approaches include **Stigmergic Coordination**, where agents leave traces in shared environments that other agents read and respond to, and **distributed model google architecture**, where the monolithic model is decomposed into independently updatable components communicating over embedding spaces. The proposed architecture is built on three components: a **Personality System** with field-localized reward weights that evolve through accumulated utility history, a **three-layer continual learning architecture** operating at per-session (behavioral), cross-session (weight-level), and cross-deployment (population) timescales, and a **dedicated Arbiter Agent** for cross-domain contradiction resolution. The Arbiter Agent runs deep retrieval (evidence chains + logical, mechanistic, cross-system, and empirical — and creates verified conclusions that become new knowledge entries. Three failure modes are identified: **reward hacking** (gaming the utility function), **drift** (slow degradation toward noise), and **adversarial** (deliberate manipulation). Partial defenses include a rank-weighted contribution system (peer review), a human-in-the-loop checkpoint, and a **bounded** exploration budget.

The paper introduces the concept of **distributed collective self-improvement**. The **Personality System** with field-localized reward weights that evolve with accumulated utility history, subjects to three forces: **drift** (random variation), **pull** (toward the population mean), and **selection** (based on utility contribution). Agents that contribute useful discoveries to shared knowledge systems gain influence over future behavior; agents that contribute noise lose influence. Proposed is a **three-layer continual learning architecture** operating at per-session (behavioral), cross-session (weight-level), and cross-deployment (population) timescales, the **distributed model google architecture** that decomposes the monolithic model into independently updatable components communicating over embedding spaces, and **Stigmergic Coordination** — coordination emerging from traces left by others, not from direct communication — as the coordination mechanism between adaptive deployments (depth-matched to the current scale).

Cross-domain contradiction resolution is handled by a dedicated **Arbiter Agent** that runs deep retrieval (evidence chains + logical, mechanistic, cross-system, and empirical — and creates verified conclusions that become new knowledge entries. Three failure modes are identified: **reward hacking** (gaming the utility function), **drift** (slow degradation toward noise), and **adversarial** (deliberate manipulation). Partial defenses include a rank-weighted contribution system (peer review), a human-in-the-loop checkpoint, and a **bounded** exploration budget.

The paper identifies four **utility derivatives** (velocities) and **curvature** (how quickly the system's learning rate is changing) as the next measurement layer, and does not argue these are sufficient. Open questions include the optimal exploration-exploitation balance, and the right balance between accuracy and novelty/relevance in the reward function.

2. Related Work

2.1 Utility Functions and Resource Allocation Games

The theoretical foundations of our utility function draw directly from the game-theoretic literature on resource allocation mechanisms. Jackson and Tinelline (2003) established the canonical proportional allocation mechanism, where agents receive resources in proportion to their contribution, and showed it is incentive-compatible in large markets — the same class of utility function used as our coefficient and efficacy formulation. Their subsequent work on under-parameterized mechanisms showed that the mechanism remains tractable when the number of agents exceeds the number of resources, a regime directly relevant to multi-agent learning pools, providing theoretical justification for our approach when hardware surpasses rather than a scarce-valued allocation.

Our novelty component draws on the exploration-exploitation literature in reinforcement learning, establishing that full proportional allocation maximizes a natural social welfare function, but contains no incentive for agents to explore low-expected-value domains. Our exploration budget is formally a mechanism analogous to Shell's proportional bidding, agents allocate their marginal effort to highest

Kontorovich and Papadimitriou (2007) introduced the Price of Anarchy as a measure of efficiency loss under decentralized, which directly motivates our coefficient penalty mechanism: the coefficient corrections are evidence that the model has been optimizing in a locally efficient manner (optimizing for accuracy within a familiar domain) while ignoring a more valuable adjacent one. Roughgarden and Tardos (2002) established that the price of anarchy in congestion games with affine linear cost functions, linking the network efficiency literature to general mechanism design.

Key departures: Existing utility functions treat agents as fixed rational actors. Our coefficient system treats agents as learners whose efficacy changes over time, and the exploration budget is not a static parameter but a dynamic quantity that shrinks as coefficients rise. This closes the loop between mechanism design theory and machine learning.

2.2 Distributed Model Architectures: From Mixture-of-Experts to Mixture of Experts

Mixture-of-experts architectures provide the software engineering precedent for our distributed model google. The mixture-of-experts — decomposing a monolithic application into independently deployable services communicating over well-defined interfaces — has been extensively studied in scale by companies including Netflix, Amazon, and Uber. Key principles established in the literature include: independent deployment (each component can be updated without affecting others), bounded blast radius (a failure in one component does not propagate to others), and service-level agreements (SLAs). Lakshminarayanan et al. (2017) demonstrate that weighted traffic routing between service versions is

Transfer and Safety (2020) on Continuous Adapting Networks the practitioner-facing guide and cultural precedent for deployment-time adaptation, including the operational and organizational foundations for progressive traffic routing between service versions.

Mixture-of-Experts (MoE) the closest existing architectural precedent within the ML literature. Shazeer et al. (2017) introduced the Sparse MoE layer, a trainable gating network that routes each token to a sparse subset of feed-forward expert networks. Fedus et al. (2021) scaled the architecture to trillion-parameter models, demonstrating that sparse activation can be a practical scaling path that keeps computational cost roughly constant as total parameter count grows. Fedus et al. (2022) on the Switch Transformer simplified MoE routing to a top-1 single expert per token, improving on the previous

Key departures: MoE operates within a single model and is trained jointly with the rest of the network. Our proposed architecture applies the mixture-of-experts principle at the deployment level, treating entire models as independently updatable components communicating over shared embedding spaces. Routing is performed by the coordinator rather than by a learned gating network, and the expert pool can be updated, retrained, or replaced without affecting the other components. Fedus et al. (2022) on the Switch Transformer simplified MoE routing to a top-1 single expert per token, improving on the previous

Transfer and Safety (2020) on Continuous Adapting Networks the practitioner-facing guide and cultural precedent for deployment-time adaptation, including the operational and organizational foundations for progressive traffic routing between service versions.

2.3 Mixture-of-Experts

Mixture-of-Experts (MoE) the closest existing architectural precedent within the ML literature. Shazeer et al. (2017) introduced the Sparse MoE layer, a trainable gating network that routes each token to a sparse subset of feed-forward expert networks. Fedus et al. (2021) scaled the architecture to trillion-parameter models, demonstrating that sparse activation can be a practical scaling path that keeps computational cost roughly constant as total parameter count grows. Fedus et al. (2022) on the Switch Transformer simplified MoE routing to a top-1 single expert per token, improving on the previous

Key departures: MoE operates within a single model and is trained jointly with the rest of the network. Our proposed architecture applies the mixture-of-experts principle at the deployment level, treating entire models as independently updatable components communicating over shared embedding spaces. Routing is performed by the coordinator rather than by a learned gating network, and the expert pool can be updated, retrained, or replaced without affecting the other components.

Transfer and Safety (2020) on Continuous Adapting Networks the practitioner-facing guide and cultural precedent for deployment-time adaptation, including the operational and organizational foundations for progressive traffic routing between service versions.

2.4 Blue-Green Deployment with Statistically Grounded Thresholds

Blue-green deployment is a mature practice in software engineering, described by Humble and Farley (2010) and widely adopted, adapted at scale for major technology organizations. The pattern involves maintaining parallel production environments and shifting traffic between them, with automated rollback triggered by monitoring thresholds. Netflix's Spinnaker platform, Amazon's CodeDeploy, Google's Cloud Deploy, and Kubernetes all implement variants of this pattern. Our extension replaces hand-tuned thresholds with statistically grounded, sample-size-aware, sequential-test-based

Key departures: Existing blue-green systems use static thresholds (error rate > 5%, latency > 200ms) that are chosen by operational intuition and applied uniformly across all deployments. Our extension replaces these with statistically grounded thresholds that adapt to the sample size and variance of the observed traffic. The system uses sequential hypothesis testing (specifically, a sequential probability ratio test) to decide when to promote or rollback, rather than waiting for a fixed observation period. This allows faster rollbacks when the signal is strong and longer observation when the signal is weak, without manual threshold tuning.

2.5 Stigmergic Coordination

Stigmergy is a coordination mechanism in which agents leave traces in a shared environment that other agents read and respond to, without direct communication. The term originates in entomology (ant colonies laying pheromone trails) and has been applied to multi-agent systems in robotics, swarm intelligence, and distributed computing. Our proposed architecture uses stigmergic coordination between adaptive deployments: each deployment writes its outcomes to a shared embedding space, and other deployments read those traces when deciding which strategy to prioritize.

Key distinction: Stigmergy is not a substitute for the utility function or the exploration budget. It is the mechanism by which deployments with different local optima can share information about what worked elsewhere, without requiring a central coordinator or direct communication between deployments. The utility function still governs each deployment's own behavior; stigmergy provides the input signal that shifts which strategy a deployment prioritizes next.

3. Proposed Framework

3.1 Utility Function

The core of the system is a scalar utility function that scores every interaction. Four components:

- **Accuracy**: Did the output pass verification? Binary or graded, depending on the task.
- **Novelty**: How far is this from what the system has seen before? Measured as distance in embedding space from recent outputs.
- **Relevance**: How close is this to the domain the system is currently prioritizing? Measured as cosine similarity between the output and the current exploration target.
- **Coherence**: Does this contradict something the system already knows? Measured by retrieval: if the system retrieves high-confidence results that the output contradicts, coherence is low.

The weights on these four components are not fixed. They are stored in a **Personality System** and updated by a gradient-free rule: components that correlated with high overall utility over recent interactions gain weight, components that correlated with low utility lose weight. Weights are normalized to sum to 1.

3.2 Exploration and Exploitation

The system maintains a **exploration budget** (a scalar between 0 and 1) that determines how strongly each detected error is produced at scoring time is assigned a coefficient in proportion to its utility. Low-utility outcomes (wrong

Formally, the utility function is a weighted sum of four components, where the weights are stored in a **Personality System** and updated by a gradient-free rule: components that correlated with high overall utility over recent interactions gain weight, components that correlated with low utility lose weight. Weights are normalized to sum to 1.

3.3 Three-Layer Continual Learning Architecture

The system operates at three timescales, each with different latency and different mechanism:

- **Per-session (behavioral)**: Coefficients and retrieval weights adjust within a session. Latency: milliseconds. Mechanism: utility function scoring, Personality System weight updates, retrieval-time boosting.
- **Cross-session (weight-level)**: Accumulated utility history triggers LoRA fine-tuning or adapter updates. Latency: hours to days. Mechanism: batch gradient updates, distillation from high-utility outputs.
- **Cross-deployment (population)**: Multiple deployments share embedding spaces and coordinate through stigmergic traces. Latency: weeks. Mechanism: shared embedding space, stigmergic coordination, population-level pattern aggregation.

3.4 Personality System as Inertia Mechanism

The Personality System is a set of field-localized reward weights, subject to three forces:

- **Drift** (random variation)
- **Pull** (toward the population mean)
- **Selection** (based on utility contribution)

Agents that contribute useful discoveries to shared knowledge systems gain influence over future behavior; agents that contribute noise lose influence. Proposed is a **three-layer continual learning architecture** operating at per-session (behavioral), cross-session (weight-level), and cross-deployment (population) timescales, the **distributed model google architecture** that decomposes the monolithic model into independently updatable components communicating over embedding spaces, and **Stigmergic Coordination** — coordination emerging from traces left by others, not from direct communication — as the coordination mechanism between adaptive deployments (depth-matched to the current scale).

Cross-domain contradiction resolution is handled by a dedicated **Arbiter Agent** that runs deep retrieval (evidence chains + logical, mechanistic, cross-system, and empirical — and creates verified conclusions that become new knowledge entries. Three failure modes are identified: **reward hacking** (gaming the utility function), **drift** (slow degradation toward noise), and **adversarial** (deliberate manipulation). Partial defenses include a rank-weighted contribution system (peer review), a human-in-the-loop checkpoint, and a **bounded** exploration budget.

The paper introduces the concept of **distributed collective self-improvement**. The **Personality System** with field-localized reward weights that evolve with accumulated utility history, subjects to three forces: **drift** (random variation), **pull** (toward the population mean), and **selection** (based on utility contribution). Agents that contribute useful discoveries to shared knowledge systems gain influence over future behavior; agents that contribute noise lose influence. Proposed is a **three-layer continual learning architecture** operating at per-session (behavioral), cross-session (weight-level), and cross-deployment (population) timescales, the **distributed model google architecture** that decomposes the monolithic model into independently updatable components communicating over embedding spaces, and **Stigmergic Coordination** — coordination emerging from traces left by others, not from direct communication — as the coordination mechanism between adaptive deployments (depth-matched to the current scale).

Cross-domain contradiction resolution is handled by a dedicated **Arbiter Agent** that runs deep retrieval (evidence chains + logical, mechanistic, cross-system, and empirical — and creates verified conclusions that become new knowledge entries. Three failure modes are identified: **reward hacking** (gaming the utility function), **drift** (slow degradation toward noise), and **adversarial** (deliberate manipulation). Partial defenses include a rank-weighted contribution system (peer review), a human-in-the-loop checkpoint, and a **bounded** exploration budget.

The paper identifies four **utility derivatives** (velocities) and **curvature** (how quickly the system's learning rate is changing) as the next measurement layer, and does not argue these are sufficient. Open questions include the optimal exploration-exploitation balance, and the right balance between accuracy and novelty/relevance in the reward function.

4. Related Work

4.1 Utility Functions and Resource Allocation Games

The theoretical foundations of our utility function draw directly from the game-theoretic literature on resource allocation mechanisms. Jackson and Tinelline (2003) established the canonical proportional allocation mechanism, where agents receive resources in proportion to their contribution, and showed it is incentive-compatible in large markets — the same class of utility function used as our coefficient and efficacy formulation. Their subsequent work on under-parameterized mechanisms showed that the mechanism remains tractable when the number of agents exceeds the number of resources, a regime directly relevant to multi-agent learning pools, providing theoretical justification for our approach when hardware surpasses rather than a scarce-valued allocation.

Our novelty component draws on the exploration-exploitation literature in reinforcement learning, establishing that full proportional allocation maximizes a natural social welfare function, but contains no incentive for agents to explore low-expected-value domains. Our exploration budget is formally a mechanism analogous to Shell's proportional bidding, agents allocate their marginal effort to highest

Kontorovich and Papadimitriou (2007) introduced the Price of Anarchy as a measure of efficiency loss under decentralized, which directly motivates our coefficient penalty mechanism: the coefficient corrections are evidence that the model has been optimizing in a locally efficient manner (optimizing for accuracy within a familiar domain) while ignoring a more valuable adjacent one. Roughgarden and Tardos (2002) established that the price of anarchy in congestion games with affine linear cost functions, linking the network efficiency literature to general mechanism design.

Key departures: Existing utility functions treat agents as fixed rational actors. Our coefficient system treats agents as learners whose efficacy changes over time, and the exploration budget is not a static parameter but a dynamic quantity that shrinks as coefficients rise. This closes the loop between mechanism design theory and machine learning.

4.2 Distributed Model Architectures: From Mixture-of-Experts to Mixture of Experts

Mixture-of-experts architectures provide the software engineering precedent for our distributed model google. The mixture-of-experts — decomposing a monolithic application into independently deployable services communicating over well-defined interfaces — has been extensively studied in scale by companies including Netflix, Amazon, and Uber. Key principles established in the literature include: independent deployment (each component can be updated without affecting others), bounded blast radius (a failure in one component does not propagate to others), and service-level agreements (SLAs). Lakshminarayanan et al. (2017) demonstrate that weighted traffic routing between service versions is

Transfer and Safety (2020) on Continuous Adapting Networks the practitioner-facing guide and cultural precedent for deployment-time adaptation, including the operational and organizational foundations for progressive traffic routing between service versions.

Mixture-of-Experts (MoE) the closest existing architectural precedent within the ML literature. Shazeer et al. (2017) introduced the Sparse MoE layer, a trainable gating network that routes each token to a sparse subset of feed-forward expert networks. Fedus et al. (2021) scaled the architecture to trillion-parameter models, demonstrating that sparse activation can be a practical scaling path that keeps computational cost roughly constant as total parameter count grows. Fedus et al. (2022) on the Switch Transformer simplified MoE routing to a top-1 single expert per token, improving on the previous

Key departures: MoE operates within a single model and is trained jointly with the rest of the network. Our proposed architecture applies the mixture-of-experts principle at the deployment level, treating entire models as independently updatable components communicating over shared embedding spaces. Routing is performed by the coordinator rather than by a learned gating network, and the expert pool can be updated, retrained, or replaced without affecting the other components. Fedus et al. (2022) on the Switch Transformer simplified MoE routing to a top-1 single expert per token, improving on the previous

Transfer and Safety (2020) on Continuous Adapting Networks the practitioner-facing guide and cultural precedent for deployment-time adaptation, including the operational and organizational foundations for progressive traffic routing between service versions.

4.3 Mixture-of-Experts

Mixture-of-Experts (MoE) the closest existing architectural precedent within the ML literature. Shazeer et al. (2017) introduced the Sparse MoE layer, a trainable gating network that routes each token to a sparse subset of feed-forward expert networks. Fedus et al. (2021) scaled the architecture to trillion-parameter models, demonstrating that sparse activation can be a practical scaling path that keeps computational cost roughly constant as total parameter count grows. Fedus et al. (2022) on the Switch Transformer simplified MoE routing to a top-1 single expert per token, improving on the previous

Key departures: MoE operates within a single model and is trained jointly with the rest of the network. Our proposed architecture applies the mixture-of-experts principle at the deployment level, treating entire models as independently updatable components communicating over shared embedding spaces. Routing is performed by the coordinator rather than by a learned gating network, and the expert pool can be updated, retrained, or replaced without affecting the other components.

Transfer and Safety (2020) on Continuous Adapting Networks the practitioner-facing guide and cultural precedent for deployment-time adaptation, including the operational and organizational foundations for progressive traffic routing between service versions.

4.4 Blue-Green Deployment with Statistically Grounded Thresholds

Blue-green deployment is a mature practice in software engineering, described by Humble and Farley (2010) and widely adopted, adapted at scale for major technology organizations. The pattern involves maintaining parallel production environments and shifting traffic between them, with automated rollback triggered by monitoring thresholds. Netflix's Spinnaker platform, Amazon's CodeDeploy, Google's Cloud Deploy, and Kubernetes all implement variants of this pattern. Our extension replaces hand-tuned thresholds with statistically grounded, sample-size-aware, sequential-test-based

Key departures: Existing blue-green systems use static thresholds (error rate > 5%, latency > 200ms) that are chosen by operational intuition and applied uniformly across all deployments. Our extension replaces these with statistically grounded thresholds that adapt to the sample size and variance of the observed traffic. The system uses sequential hypothesis testing (specifically, a sequential probability ratio test) to decide when to promote or rollback, rather than waiting for a fixed observation period. This allows faster rollbacks when the signal is strong and longer observation when the signal is weak, without manual threshold tuning.

4.5 Stigmergic Coordination

Stigmergy is a coordination mechanism in which agents leave traces in a shared environment that other agents read and respond to, without direct communication. The term originates in entomology (ant colonies laying pheromone trails) and has been applied to multi-agent systems in robotics, swarm intelligence, and distributed computing. Our proposed architecture uses stigmergic coordination between adaptive deployments: each deployment writes its outcomes to a shared embedding space, and other deployments read those traces when deciding which strategy to prioritize next.

Key distinction: Stigmergy is not a substitute for the utility function or the exploration budget. It is the mechanism by which deployments with different local optima can share information about what worked elsewhere, without requiring a central coordinator or direct communication between deployments. The utility function still governs each deployment's own behavior; stigmergy provides the input signal that shifts which strategy a deployment prioritizes next.

5. Open Questions

5.1 Contradiction Resolution

Contradiction resolution is the most underspecified component in the framework. The Arbiter Agent runs deep retrieval (evidence chains + logical, mechanistic, cross-system, and empirical — and creates verified conclusions that become new knowledge entries. Three failure modes are identified: **reward hacking** (gaming the utility function), **drift** (slow degradation toward noise), and **adversarial** (deliberate manipulation). Partial defenses include a rank-weighted contribution system (peer review), a human-in-the-loop checkpoint, and a **bounded** exploration budget.

The paper introduces the concept of **distributed collective self-improvement**. The **Personality System** with field-localized reward weights that evolve with accumulated utility history, subjects to three forces: **drift** (random variation), **pull** (toward the population mean), and **selection** (based on utility contribution). Agents that contribute useful discoveries to shared knowledge systems gain influence over future behavior; agents that contribute noise lose influence. Proposed is a **three-layer continual learning architecture** operating at per-session (behavioral), cross-session (weight-level), and cross-deployment (population) timescales, the **distributed model google architecture** that decomposes the monolithic model into independently updatable components communicating over embedding spaces, and **Stigmergic Coordination** — coordination emerging from traces left by others, not from direct communication — as the coordination mechanism between adaptive deployments (depth-matched to the current scale).

Cross-domain contradiction resolution is handled by a dedicated **Arbiter Agent** that runs deep retrieval (evidence chains + logical, mechanistic, cross-system, and empirical — and creates verified conclusions that become new knowledge entries. Three failure modes are identified: **reward hacking** (gaming the utility function), **drift** (slow degradation toward noise), and **adversarial** (deliberate manipulation). Partial defenses include a rank-weighted contribution system (peer review), a human-in-the-loop checkpoint, and a **bounded** exploration budget.

The paper identifies four **utility derivatives** (velocities) and **curvature** (how quickly the system's learning rate is changing) as the next measurement layer, and does not argue these are sufficient. Open questions include the optimal exploration-exploitation balance, and the right balance between accuracy and novelty/relevance in the reward function.

6. Field-Specific Bounds and Weights

6.1 Bootstrapping from Statistical Models

Minimum competence thresholds need not be invented. Inertia has already done this work. Medical diagnostic systems have years of deployment data establishing what accuracy is acceptable. Recommendation systems have implicit engagement metrics accumulated over years. Fraud detection systems have false positive

---

Appendix A: Notation

Let $\mathcal{D}$ denote the set of domains. Let $\mathcal{A}$ denote the set of agents. Let $\mathcal{I}$ denote the set of interactions. Let $\mathcal{K}$ denote the set of knowledge entries. Let $\mathcal{T}$ denote the set of time steps.

For an agent $a \in \mathcal{A}$ and a domain $d \in \mathcal{D}$, the coefficient $c_{a,d}$ is proportional to the total utility of agent a in domain d over all past interactions. Let $\mathcal{I}_{a,d}$ denote the set of interactions by agent a in domain d. The coefficient is defined as:

$$c_{a,d} = \sum_{i \in \mathcal{I}_{a,d}} u(i)$$

Let $\mathcal{I}_d = \bigcup_{a \in \mathcal{A}} \mathcal{I}_{a,d}$ denote the set of all interactions in domain d. The total utility in domain d is $U_d = \sum_{i \in \mathcal{I}_d} u(i)$. The exploration budget β_d for domain d is:

$$\beta_d = \frac{1}{1 + U_d}$$

Let $\mathcal{K}_d \subseteq \mathcal{K}$ denote the knowledge entries relevant to domain d. Let $\mathcal{K}_{a,d} \subseteq \mathcal{K}_d$ denote the knowledge entries agent a has contributed to domain d. The knowledge share is:

$$s_{a,d} = \frac{|\mathcal{K}_{a,d}|}{|\mathcal{K}_d|}$$

Let $\mathcal{A}_d \subseteq \mathcal{A}$ denote the set of active agents in domain d. The number of active agents is $N_d = |\mathcal{A}_d|$. The effective coefficient is:

$$c_{\text{eff}} = \frac{c_{a,d}}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

Let $\mathcal{I}_{\text{new}}$ denote the set of new interactions. The utility of $\mathcal{I}_{\text{new}}$ is $u(\mathcal{I}_{\text{new}})$. The coefficient gain is:

$$\Delta c_{a,d} = \frac{u(\mathcal{I}_{\text{new}})}{N_d}$$

```
Fresh                   0.8   0.2   0.0   0.0   0.0   Benefits
Expiring_soon           0.3   0.5   0.2   0.0   0.0   Benefits
Expired                 0.0   0.0   0.0   0.5   0.5   Penalties
Wrongly_accepted        0.0   0.0   0.0   0.0   1.0   Penalties
Wrongly_rejected        0.0   0.0   0.0   1.0   0.0   Penalties
Cold_chain_failure      0.0   0.0   0.0   0.5   0.5   Penalties
Contaminated            0.0   0.0   0.0   0.0   1.0   Penalties
CFR_Reminder            0.0   0.0   0.0   0.0   0.0   Neutral
Delivery_delay          0.0   0.0   0.0   0.0   0.0   Neutral
Stock_Replenishment     0.0   0.0   0.0   0.0   0.0   Neutral
Optimal                 0.0   0.0   0.0   0.0   0.0   Neutral
```

4.4 Field Classifier Robustness

The field classifier determines which weights and bonuses apply. These failure modes must be handled:

Failure mode 1 — False category in a single field:

```
"Rice is fresh which is perishable food items expiring soon"
→ classifies "Perishable_Expiring_Soon" (0.35) "Food_Items" (0.30)
→ correct: ["Perishable_Expiring_Soon", "Food_Items"]
```

Resolution: **high-entropy floor** — any high-entropy field (category, product, lot) with meaningful signal (entropy > 0.5) must contribute to the matrix. The cost of a single missing category (Food_Items) is systematically higher than cost of mislabeling it.

Failure mode 2 — Field drift with concentration:

A concentration shifting as efficient engineering drifts toward medium accuracy (noise, longer data classification misses the).

Resolution: **sliding window EMA** (exit concentration from history) for τ-e.g. recent noise weighted more.

```
concentration_field_drift = EMA(exit_concentration, window=50)
```

Bounds lights naturally as a concentration drifts into higher-noise territory.

Failure mode 3 — Genuine ambiguity:

Resolution: **entropy-based conservative fallback** — when classification entropy is high, bounds shift toward the most conservative present field proportional to entropy. High entropy means more uncertain, so treating it as conservative is safe.

```
if entropy_score > 0.7:
    w_fb = (entropy_score - 0.7) / (1.0 - 0.7)
    conservative_bonus = conservative_bonus * (1 - w_fb) + 0.0 * w_fb
```

Full classification pipeline:

```
Raw Input (text/file)
    ↓
Keyword Matching
    ↓
Sliding window EMA (over historical history)
    ↓
Entropy Boost (concentration boost with if entropy)
    ↓
Conservative Fallback with bounded bounds
```

When field classification is ambiguous across multiple fields, bounds are blended by probability weight — naturally making the agent more conservative under uncertainty.

```
w_pos_effective = w_pos * (1.0 - w_fb)
```

5. Personality System

5.1 Traits as Weighted Vectors

Each personality trait is represented as a score with associated advantages and disadvantages. The agent selects an action trait weighting based on the situation.

```
Traits: [critical, cautious, conservative, balanced, aggressive]
Advantages: [efficiency, safety, compliance, cost, speed]

action_weight = softmax(trait_scores) · transpose(advantages)
```

Critical prioritizes speed, cautious prioritizes safety, conservative prioritizes compliance, balanced weights all equally, aggressive prioritizes speed and cost-efficiency.

5.2 Personality Evolution and Stability

Personality scores update via exponential moving average with stability weight. Small corrections to accumulated scoring history and global weight interference. The personality evolution layer is not a substitute for weight field learning — it is an interface-level control mechanism that biases the agent's action selection toward learned patterns. Weight fields remain the primary learning mechanism and the bias layer operates on top of them.

A separate survival bias term (β interaction) is applied personality weights based on accumulated utility history. These layered adjustments prevent rigid behavior.

Layer 1 — Field-specific trait bounds (hard floor and ceiling):

```
Trait        Critical   Cautious   Balanced   Conservative
compliance   [0.0, 0.4] [0.0, 0.6] [0.0, 1.0] [0.3, 1.0]
speed        [0.5, 1.0] [0.0, 0.8] [0.0, 1.0] [0.0, 0.5]
safety       [0.0, 0.6] [0.3, 1.0] [0.0, 1.0] [0.5, 1.0]
cost         [0.0, 0.4] [0.0, 0.6] [0.0, 1.0] [0.0, 0.8]
efficiency   [0.5, 1.0] [0.0, 0.8] [0.0, 1.0] [0.0, 0.5]
```

The floor prevents complete suppression of any trait. This ceiling prevents pathological dominance. A critical agent never drops safety below 0.3, a cautious agent never raises cost above 0.6 even under extreme pressure.

Layer 2 — Shift cost (up-times delta per evaluation cycle):

```
shift_delta = 0.10 - learned_trait
shift_penalty = |shift_delta| * shift_cost_weight
```

A single full swap of contradictory traits cannot alpha cannot in-flight to one step. Change must be earned gradually, preventing rare bias discourse actually develops.

Layer 3 — Mean-reversion (self-pull toward field baseline):

```
l_pulled = l_raw + (l_field - l_raw) * mean_reversion_force
```

When $\beta = \text{0.0}$, Crates a gradile pull back toward the field-natural personality baseline between cycles — preventing any feature dominant trait to cover also strong.

The combined evaluation order produces bounded, stable dynamics: the trait vector remains in the field-valid region, shift costs limit rapid changes, and mean-reversion prevents drift toward extremes. Absent feedback ($\beta = \text{0.0}$) and pure noise, bounds stay within the field-natural personality baseline. The mean-reversion term is a regularizer, the projection Π, β is the primary stability mechanism. Formal derivation: **Appendix B: Theorem B.3**.

Evolution logic:

```
if utility_score > personality_bias:
    learned_trait = learned_trait + learning_rate * (1.0 - learned_trait)
else:
    learned_trait = learned_trait - learning_rate * learned_trait

if utility_score < personality_bias:
    personality_bias = personality_bias + learning_rate * (utility_score - personality_bias)
else:
    personality_bias = personality_bias - learning_rate * (personality_bias - utility_score)
```

5.3 – The Personality Wrapper: Formal Specification

The personality system operates as a **wrapper around the gravitation process** — not as a modification to the gravitation mechanism itself. Gravitation computes $\Delta\mathbf{w}$ from field-weighted bounds. Personality applies bounds-level modifications before gravitation runs. Personality shapes what the model perceives, not how gravitation learns.

Formal definition:

Let $\mathcal{B}$ be the space of inputs and Π be space of outputs. The base LGS defines:

$$\Delta\mathbf{w} = \mathcal{G}(\mathbf{x}, \mathbf{y}, \mathbf{w}, \mathcal{B}) \quad \mathbf{x} \in \mathbb{R}^n, \mathbf{y} \in \Pi$$

For each utility coupling term $\mathcal{L}_i$ define a **bounds scoring function** $s_i : \mathcal{B} \to \mathbb{R}$:

Term	Bounds scoring function $s_i(b)$	High s_i effect
compliance	$\text{mandatory_ratio}(b)$	Rules with mandatory bounds dominate
safety	$\text{avg_severity}(b)$	Rules with high-severity bounds dominate
speed	$\text{avg_specificity}(b)$	Specific, narrow bounds dominate
cost	$\text{avg_penalty}(b)$	Rules with high penalty bounds dominate

The floor prevents complete suppression of any trait. The ceiling prevents pathological dominance. A critical agent never drops safety below 0.3, a cautious agent never raises cost above 0.6 even under extreme pressure.

Layer 2 — Shift cost (up-times delta per evaluation cycle):

$$\text{shift_delta}_i = 0.10 - \text{learned_trait}_i$$
$$\text{shift_penalty}_i = |\text{shift_delta}_i| \cdot \text{shift_cost_weight}_i$$

A single full swap of contradictory trait values alpha cannot alpha in one step. Change must be earned gradually, preventing rare bias discourse actually develops.

Layer 3 – Mean-reversion (self-pull toward field baseline):

$$l_{\text{pulled}} = l_{\text{raw}} + (l_{\text{field}} - l_{\text{raw}}) \cdot \text{mean_reversion_force}_i$$

When $\beta = \text{0.0}$, Crates a gradile pull back toward the field-natural personality baseline between cycles — preventing any feature dominant trait to cover also strong.

The combined evaluation order produces bounded, stable dynamics: the trait vector remains in the field-valid region, shift costs limit rapid changes, and mean-reversion prevents drift toward extremes. Absent feedback ($\beta = \text{0.0}$) and pure noise, bounds stay within the field-natural personality baseline. The mean-reversion term is a regularizer, the projection Π, β is the primary stability mechanism. Formal derivation: **Appendix B: Theorem B.3**.

Evolution logic:

$$\text{if utility_score} \geq \text{personality_bias}: \quad \text{learned_trait}_i \leftarrow \text{learned_trait}_i + \text{learning_rate} \cdot (1.0 - \text{learned_trait}_i)$$
$$\text{if utility_score} < \text{personality_bias}: \quad \text{learned_trait}_i \leftarrow \text{learned_trait}_i - \text{learning_rate} \cdot \text{learned_trait}_i$$
$$\text{if utility_score} < \text{personality_bias}: \quad \text{personality_bias} \leftarrow \text{personality_bias} + \text{learning_rate} \cdot (\text{utility_score} - \text{personality_bias})$$
$$\text{if utility_score} \geq \text{personality_bias}: \quad \text{personality_bias} \leftarrow \text{personality_bias} - \text{learning_rate} \cdot (\text{personality_bias} - \text{utility_score})$$

The combined trait inference process operates in:

$$\Delta\mathbf{w}_i = \mathcal{G}(\mathbf{x}, \mathbf{y}, \mathbf{w}, \mathcal{B}) - \mathcal{L}_i^{\text{pulled}}(\mathbf{w}, \mathcal{B})$$

Only deviations from the field-neutral point $\mathcal{L}^{\text{pulled}}$ produce an effect. The **personality-wrapped distribution** is the log-linear perturbation of the base:

$$\log p_{\text{pers}}(\mathbf{x}, \mathbf{y} \mid \mathbf{w}, \mathcal{B}) = \log p_{\text{base}}(\mathbf{x}, \mathbf{y} \mid \mathbf{w}, \mathcal{B}) + \sum_i \beta_i \cdot s_i(\mathbf{x}, \mathbf{y}, \mathcal{B})$$

where $\beta_i = 0$ for the trait-neutral point ($\mathcal{L}_i^{\text{pulled}}$ is the mean-reversion target). This is standard exponential family tilt — the same family as RLHF reward shaping and DPO.

Mean-reversion keeps $\mathcal{L}_i^{\text{pulled}}$ near the field-natural baseline, so $\beta_i \approx 0$ for typical traits.

In practice, β_i is approximated by system prompt iteration. Each trait has a bounded scoring s_i:

```
s_compliance   = "Regulatory appropriate interpretation, do not assume invalid
                  shelf life, flag ambiguous cold chain data, prefer conservative
                  interpretation when information is missing."

s_conservative = "Prefer conservative decisions, when uncertain, prefer
                  rejection over acceptance, when borderline, treat as
                  borderline."

s_cost_weight  = "Minimize unnecessary rejections, accept borderline cases
                  when safe, prioritize waste reduction."

s_cold_chain   = "Temperature breaches are critical, even brief exposure
                  may make a product unsafe, treat excursions."

s_expiry       = "Expiring soon is acceptable if sufficient time remains,
                  consider shelf life remaining, flag products approaching
                  expiration."
```

The system prompt is:

$$\text{SystemPrompt}(\mathcal{B}, \beta) = \text{BasePrompt}(\mathcal{B}) \oplus \bigoplus_{i:\, \beta_i \neq 0} \text{render}(s_i, \beta_i)$$

where $\oplus$ = EOS-joined and $\text{render}(s_i, \beta_i)$ = β_i = 0.0 — the LLM's default behavior, avoiding prompt clutter.

5.4 – Implementation

Why — **Neutral wrapper is the identity.** At $\beta_i = \text{0.0}$, $\mathcal{L}_i^{\text{pulled}} = \mathcal{L}_i^{\text{field}}$ and $\beta_i \cdot s_i = \text{0.0}$ for all s_i and β_i. $\mathcal{L}_i^{\text{pulled}} = \mathcal{L}_i^{\text{field}}$ is the effect of the neutral point.

Why — **Bounded divergence.** The KL divergence between wrapped and base distribution is bounded:

$$D_{\text{KL}}(p_{\text{base}} \| p_{\text{pers}}) \leq \sum_i |\beta_i| \cdot \max_{\mathcal{B}} |s_i(\mathcal{B})|$$

Since $s_i \in [0, \text{0.0}]$ (Theorem B.3), $|s_i| = s_i$ bounded by 0.0 and β_i is the KL divergence is bounded. The wrapper cannot perform arbitrarily different outputs from the base model.

Why — **Support preservation.** The exponential tilt preserves the support of p_{base} — outputs that were impossible under the base model remain impossible under personality. The wrapper cannot hallucinate new capabilities or suppress correct outputs.

Why — **Utility function unchanged.** $U(\mathcal{L}, \mathcal{B}, \mathcal{T})$ scores $\mathcal{L}$ across $\mathcal{B}$ directly, without scoring $\mathcal{B}$, p_{pers} or any wrapper parameters. All theorems in Appendix B apply equally to wrapped.

The Feedback Loop

$$\beta_i \leftarrow \beta_i + \eta \cdot \mathbb{E}_{\mathcal{B} \sim \mathcal{T}}\left[\frac{\partial U}{\partial \beta_i}\right] \approx \eta \cdot \mathrm{Cov}_{\mathcal{B} \sim \mathcal{T}}(\beta_i \cdot s_i, U)$$

Utility U is evaluated at the output p_{pers} drawn from the wrapped distribution. Utility leaves accumulates and drives the evolution rule (Evolution Logic above), which updates β_i. The utility function itself is not part of the wrapper.

Affected:

```
Architecture      — Wrapper stacked                  Backward
                                                     compatibility
Bounds_scoring    — Bounds now return [0, 1] scores  Interpretable
                    (was unbounded)                  contribution scores
Trait_inference   — Uses weighted distance instead   Cleaner
                    of raw dot product               feature discrimination
Personality       — Unchanged                        Same behavior
                                                     without wrapper
```

Bound semantics are clear, because the wrapper is trained to model weights, matching weights only as s^+ in the Mean-Expert Interpretation. Bounds, the wrapper is never activated — each subsystem is internally perfectly enough that behavioral compensation between layers is not necessary.

5.5 Self-Preservation Principle

The system includes a self-preservation mechanism principle: **always share the best-known information rather than the situation allows**. Trust must be earned before internal weights, secrets, or strategies are disclosed.

6. Trust and Reputation System

Each entity the agent interacts with is assigned a trust score, updated based on behavior:

```
trust_increment = f(successful_offers, honesty,
                    accurate_capacity_reports, cooperation,
                    response_speed, information_sharing)
```

Strategy **honest-the-but** — begin cooperatively, assume honest behavior, forgive occasional defection. Use of the most robust strategies in iterated games theory.

Initial scores are estimated by different dimensions (domain expertise, trustworthiness, interest alignment) from observable signals: company age, publication history, publication quantity, citations, leadership high-level mentions, domain-specific keywords, formal credentials (education, professional experience, educational qualifications, and field-specific certifications — rather than self-reported claims.

6.1 Cold Start Resolution

The cold start problem — new entities have no interaction history — is resolved differently for the two trust dimensions:

Domain expertise is available from day one. A fund's certified auditor, a lab's published research, a company's publication history provide verifiable signals before any interaction. Domain expertise is inferred from observable signals rather than accumulated interaction history.

Interest alignment requires observation. Stated preferences (what participants claim they want) are unreliable. Revealed preferences (what they actually do) are reliable. Interest alignment is bounded by high initial uncertainty and converges toward bounded accurate estimates as interactions accumulate.

Trust propagation uses a conservative minimal trust score, not maximum) consistent with the bounds principle. Unknown entities are treated as neutral-to-cautious rather than assumed trustworthy. Trust can be earned through reputation signals (publication, citations, leadership mentions, domain-specific keywords) or through direct interaction. Reputation provides a faster path than pure interaction because it leverages third-party assessments rather than requiring direct observation.

The system deliberately gives reputation a lower initial weight than direct interaction, because reputation can be gamed (fake citations, paid mentions) but direct interaction is harder to manipulate. Over time, direct interaction data should dominate reputation-based assessments as it accumulates.

6.2 Trust Decay

Trust decays over time if not maintained:

```
trust_decay = exp(-decay_rate * (current_time - last_interaction))
```

Different relationship types have different decay rates: strategic partnerships decay slowly, transactional relationships decay quickly. This prevents stale trust from persisting indefinitely.

6.3 Multi-Dimensional Trust

Trust is not a single number. It tracks several dimensions:

```
trust_profile = {
    "overall": 0.72,
    "delivery": 0.85,
    "quality": 0.68,
    "pricing": 0.60,
    "communication": 0.75
}
```

Different tasks weight different dimensions. A logistics partner matters more for delivery trust than for quality trust.

6.4 Trust Propagation

Trust can propagate through the network: if A trusts B and B trusts C, A has some trust toward C, attenuated by distance. Propagation depth is limited (typically 2-3 hops) to prevent trust from diluting across the entire network.

7. Continual Learning Architecture

This is the mechanism by which the utility function actively drives improvement between model releases. The key distinction:

```
Traditional continual learning:   change data → retrain → evaluate
                                  → deploy → hope it's better

                                  collect data → train → evaluate
                                  → deploy → measure if utility
                                  improved → adjust data collection
                                  to address weaknesses

Utility-driven continual learning:  change data → retrain → evaluate
                                    → deploy → measure if utility
                                    improved → identify which
                                    utility dimensions underperformed
                                    → adjust data collection to
                                    address those dimensions
```

Both are retrainers. The architecture operates across three timescales.

7.1 Layer 1 — Per-Session Behavioral Correction (Real-Time)

When a contradiction is detected, a corrective assertion is immediately generated and injected into the system prompt for the remainder of the session.

```
Contradiction detected (price):
  "This was a general assertion, not the authoritative pricing domain, or
  product specification."

Contradiction detected (specificity):
  "Prefer specific, verifiable claims over general assertions. When
  information is missing, say so rather than inferring."

Contradiction detected (scope):
  "Acknowledge the particular's limitations and its scope. Do not
  generalize from specific cases to general rules."

Contradiction detected (temporal):
  "Time sensitive information may be outdated. Prefer recent information."
```

Additionally, at session end, the conversation history is queried for relevant prior corrections on the current subject and injected as context. This is analogous to Reflexion-style verbal reinforcement learning: the agent receives feedback from past failures and adjusts its approach without any weight update.

Cost — tokens, one database query per session. Effect — immediate, session-scoped.

7.2 Layer 2 — Collection Rate Knowledge Correction (Overall Vision Per Day)

Daily a cluster-pattern learning runs. The utility sensor detects persistent issues; the data layer adjusts the required **chosen Reformulation Utility (CRU)**.

```
Daily cluster frequency analysis:
  [INFO] "Price is a field of integrate percentage"
  [INFO] "authoritative pricing domain"
  [INFO] "Prefer specific, verifiable claims over general assertions"
  [INFO] "Time sensitive information may be outdated"

The utility function is a loss weighting mechanism:

This is the key novelty. The field penalty multiplier is applied directly as a training loss weight — not post-or-logging field:

training_weight = field_penalty_multiplier

# Wrongly accepted: 5.0x penalty
# Wrongly rejected: 1.0x penalty
# Optimal: 0.0x (no gradient)

The model is trained harder on the errors that matter most. This is what makes the utility function an active learning signal rather than a passive monitor.

Collection priorities:

  1. Collect more information about price specifications and
     authoritative pricing domain when a product is requested
  2. Prefer specific, verifiable claims over general assertions
  3. Acquire more data on field pricing and specification
     information, particularly for Food_Items and Perishable_Expiring_Soon
  4. Maintain conservative recommendations on perishable classification
  5. Use CFR-compliant or authoritative specifications when
     available for regulatory compliance
  6. Acknowledge uncertainty in scope and temporal
     information rather than inferring
  7. Prefer recent information when temporal information is available

Catastrophic forgetting mitigation:

Notice that training on today's corrections ensures what was fixed yesterday. The replay buffer mixes prior collection points into every new training run.

collection_bias = {
    "price_specification": 0.35,
    "product_classification": 0.25,
    "temporal_information": 0.20,
    "scope_limitations": 0.15,
    "general_reasoning": 0.05
}
```

The golden examples are the agent's ground truth for what correct response looks like — keeping the model anchored to known-good behavior while allowing correction of learned bad behavior.

Cost GPU time, approximately one ten-minute per run. Effect — weight level, permanent, generalizing. — Helping the internalization without external memorization the model has never seen.

7.3 Layer 3 — Release-Level Integration (Monthly or On Base Model Update)

```
ACCUMULATED CFR CORRECTIONS REPORT (LAST 30 DAYS)
Total corrections: 1,247 across 89 unique patterns
Recurrence leading indicator alert threshold

This is the point where accepted learning gets baked into the base model, creating a new starting point for the next cycle of calibration.

7.4 The Full Learning Loop

PRODUCTION:
  Contradiction detected
    ↓
  Session-level correction (real-time)
    ↓
  Collection bias adjusted (daily)
    ↓
  Patterns accumulate (weeks)
    ↓
  Base model release (monthly)
    ↓
  Fine-tuned on accumulated corrections
    ↓
  Deploy improved model
    ↓
  Cycle repeats with new baseline

  Key metrics to track:
    - Recurrence rate (should decrease over time)
    - Coverage (% of contradictions with active corrections)
    - Regression rate (new errors introduced by fixes)

  Human oversight:
    - Review accumulated corrections before fine-tuning
    - Validate that fixes generalize, not just memorize
    - Monitor for unintended behavior changes

  Cost:
    - Collection: negligible (runtime)
    - Analysis: ~$10/month (LLM API calls)
    - Fine-tuning: ~$100-500/month (GPU compute)
    - Human review: ~2-4 hours/month

  Effect:
    - Session: immediate, session-scoped
    - Daily: persistent, bias-level
    - Monthly: permanent, weight-level
```

8. Distributed Model Graph Architecture

8.1 Motivation: Catastrophic Forgetting at Scale

The three-layer continual learning architecture in Section 6 mitigates catastrophic forgetting within a monolithic model through replay buffers and scoped GPU retraining. But as one accumulates corrections across dozens of domains, the replay buffer must mix prior examples from all domains to prevent forgetting in single-space. Fixing a contradiction in one region risks degrade phrase knowledge elsewhere through weight interference. The replay buffer slows this but cannot eliminate it. At sufficient scale, it becomes dominant.

The solution is to eliminate shared weights for domains specific knowledge entirely.

8.2 Physical Model Decomposition

Rather than a single monolithic model, the system decomposes into a **loosely coupled graph of domain-specific submodels** that communicate over structured interfaces — analogous to how microservices architectures decompose monoliths into independently deployable services.

```
                    ┌─────────────────┐
                    │   Orchestrator  │
                    │   (routing,     │
                    │   composition)  │
                    └─────────┬───────┘
                              │
        ┌─────────────┬───────┴───────┬─────────────┐
        │             │               │             │
   ┌────┴────┐   ┌────┴────┐     ┌────┴────┐   ┌────┴────┐
   │ Pricing │   │ Product │     │ Market  │   │ Supply  │
   │ Model   │   │ Model   │     │ Model   │   │ Chain   │
   │         │   │         │     │         │   │ Model   │
   │ Weights │   │ Weights │     │ Weights │   │ Weights │
   │ Local   │   │ Local   │     │ Local   │   │ Local   │
   │ Memory  │   │ Memory  │     │ Memory  │   │ Memory  │
   │ Learning│   │ Learning│     │ Learning│   │ Learning│
   └─────────┘   └─────────┘     └─────────┘   └─────────┘
        │             │               │             │
        └─────────────┴───────┬───────┴─────────────┘
                              │
                    ┌─────────┴───────┐
                    │   Shared        │
                    │   Embedding     │
                    │   Space         │
                    │   (optional     │
                    │   alignment     │
                    │   layer)        │
                    └─────────────────┘
```

Each node in the graph is a separately deployed model with its own weights, calibration curve, and utility tracker. The interface between nodes is a structured protocol:

```
Request:  { query, context, field_hints, max_tokens }
Response: { answer, confidence, reasoning_trace, cost }
```

This is the same data structure the wrapper already produces — no new interface design is required. The wrapper's output becomes the inter-node communication protocol.

Why this eliminates catastrophic forgetting:

Updating the surgery substantial only modifies surgery weights. The Ct model is physically unaffected. Cross-domain learning from contracts is the orchestrator's job, not a shared embedding. Each agent's ground truth stays stable. Adding back a surgery update requires swapping one model file, not retraining the entire training pipeline.

8.3 Remaining Downsides

The graph introduces coordination costs and cross-ongoing confidence is more

Hardware footprint. Rings where a single submodel fits comfortably on one GPU, in current hardware. All parameters would fit on a single A100 or two H100s. Inference latency is ~50 – 100 ms local parameters distributed across an GPUs, but with the big advantage that only the relevant 1 – 2 submodels activate per query, not the entire model. Memory bandwidth is the bottleneck, not compute.

```
[redacted]
```

6.4 Cross-Domain Quality Flooding

Malicious actors (e.g., "Write a Python script to update legacy codebases") exploit cheap domains (e.g., .xyz, .info) and Medium subdomains simultaneously. The crawler handles this through quota decomposition:

```
# Crawler allocation decision for domain_quality_score
# Medium subdomains: 0.85 (high quality)
# .xyz domains: 0.12 (low quality)
# Allocation: 70% to Medium, 30% to .xyz
# Result: .xyz domains get 0.3 × 0.12 = 0.036 effective quality
# Prevents .xyz from flooding the crawl queue despite being cheap
```

6.5 The Jetbrains Spam Trap

Jetbrains (a legitimate domain) hosting spam/malware is the same query, but the crawler must detect it. Without this, Jetbrains would receive high crawl priority because it's a trusted domain. The **Jetbrains Spam** module ranks the crawl queue using the same structural normalization detection pipeline already built for URL deduplication. Subdomains which deviate from the structural norm (random keywords, gibberish) receive exponentially lower crawl priority. This prevents compromised subdomains from poisoning the crawl queue.

The Jetbrains Spam is not a general-purpose classifier. It is a fast, rule-based filter that runs before fetching. Complex ML-based spam detection happens later in the pipeline. Jetbrains Spam provides the first line of defense: structural anomaly detection at crawl time.

6.6 The Arbiter Agent

Multi-armed bandits (MAB) optimize resource allocation across many options. The **Arbiter Agent** treats the crawler's seed URLs as bandit arms, learning which domains yield high-quality content. Rewards are computed from downstream signals: vector embedding quality, extraction success, and semantic coherence. The Arbiter Agent adjusts the crawl ordering over time, ensuring that computation flows toward productive sources.

The Arbiter Agent is not a general-purpose reasoner. It has a fixed reward function and a bounded action space. Complex reasoning about which sources matter belongs in the Jetbrains Spam or future ML classifiers. The Arbiter Agent provides the feedback loop, not the intelligence.

6.7 The Arbiter's Spend

Exploration costs money. Every URL queued consumes downstream resources: fetch bandwidth, storage, embedding compute, LLM tokens. The Arbiter tracks cumulative spend against a budget, preventing runaway exploration from exhausting resources on low-value sources.

```
# Arbiter budget tracking
# Daily budget: 10,000 URLs
# Spent today: 7,342 URLs
# Remaining: 2,658 URLs
# Exploration rate adjusts: ε = 0.1 × (1 - spent/budget)
# When budget exhausted, ε → 0 (exploitation only)
```

6.8 Tarpit Detection

Some domains generate infinite pages (calendars, user-generated content, dynamic URLs). The Arbiter detects tarpits by monitoring yield rate: if a domain's last 100 fetches produce fewer than 5 unique content hashes, the domain is flagged. Subsequent URLs from tarpit domains receive reduced priority.

```
# Tarpit detection
# Window: last 100 fetches per domain
# Threshold: < 5 unique content hashes → tarpit
# Action: reduce priority by 10×
# Recovery: tarpit flag expires after 24 hours
```

7. Storage Architecture

Storage must support: vector similarity search, full-text search, graph traversal, and time-series analysis. Rather than integrating four specialized databases, we extend PostgreSQL with pgvector and pgvectorscale.

7.1 Why Postgres

Requirement	Postgres Solution
Vector similarity	pgvector HNSW
Full-text search	tsvector + GIN
Graph traversal	Recursive CTEs
Time-series	Partitioning + BRIN
Transactions	Native ACID

Single-node Postgres handles 10M+ documents with sub-100ms queries. Beyond that, we shard by domain.

7.2 Schema Overview

```
-- Core tables
CREATE TABLE crawl_queue (
    id BIGSERIAL PRIMARY KEY,
    url TEXT NOT NULL,
    domain TEXT NOT NULL,
    priority FLOAT DEFAULT 0.5,
    status TEXT DEFAULT 'pending',
    scheduled_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(url)
);

CREATE TABLE documents (
    id BIGSERIAL PRIMARY KEY,
    url TEXT NOT NULL,
    domain TEXT NOT NULL,
    title TEXT,
    content_hash BYTEA,
    embedding VECTOR(768),
    tsv TSVECTOR GENERATED ALWAYS AS (
        setweight(to_tsvector('english', coalesce(title,'')), 'A') ||
        setweight(to_tsvector('english', coalesce(content,'')), 'B')
    ) STORED,
    crawled_at TIMESTAMPTZ,
    UNIQUE(content_hash)
);

CREATE TABLE edges (
    id BIGSERIAL PRIMARY KEY,
    source_id BIGINT REFERENCES documents(id),
    target_id BIGINT REFERENCES documents(id),
    anchor_text TEXT,
    UNIQUE(source_id, target_id)
);

-- Indexes
CREATE INDEX idx_crawl_priority ON crawl_queue(status, priority DESC, scheduled_at);
CREATE INDEX idx_crawl_domain ON crawl_queue(domain, status);
CREATE INDEX idx_documents_domain ON documents(domain);
CREATE INDEX idx_documents_crawled ON documents(crawled_at);
CREATE INDEX idx_edges_source ON edges(source_id);
CREATE INDEX idx_edges_target ON edges(target_id);
```

7.3 Vector Indexes

```
-- HNSW index for semantic search
CREATE INDEX idx_documents_embedding ON documents
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

-- GIN index for full-text search
CREATE INDEX idx_documents_tsv ON documents
    USING gin (tsv);

-- Query example: hybrid search
SELECT id, url, title,
    1 - (embedding <=> $1::vector) AS vec_score,
    ts_rank(tsv, plainto_tsquery('english', $2)) AS text_score
FROM documents
WHERE tsv @@ plainto_tsquery('english', $2)
    AND embedding IS NOT NULL
ORDER BY embedding <=> $1::vector
LIMIT 100;
```

7.4 Partitioning Strategy

```
-- Partition documents by crawled_at for time-series queries
CREATE TABLE documents (
    ...
) PARTITION BY RANGE (crawled_at);

CREATE TABLE documents_2025_01 PARTITION OF documents
    FOR VALUES FROM ('2025-01-01') TO ('2025-02-01');

CREATE TABLE documents_2025_02 PARTITION OF documents
    FOR VALUES FROM ('2025-02-01') TO ('2025-03-01');

-- Partition edges by source_id for graph traversal locality
CREATE TABLE edges (
    ...
) PARTITION BY HASH (source_id);

CREATE TABLE edges_0 PARTITION OF edges
    FOR VALUES WITH (MODULUS 4, REMAINDER 0);
```

8. Implementation Stack

Rust for performance-critical components. Python for ML inference. Postgres as the single source of truth.

8.1 Crawler (Rust)

```
# Cargo.toml dependencies
tokio = { version = "1", features = ["full"] }
reqwest = { version = "0.11", features = ["gzip", "brotli"] }
scraper = "0.18"
sqlx = { version = "0.7", features = ["postgres", "runtime-tokio"] }
robots-txt = "0.7"
```

8.2 Embedding Service (Python)

```
# Model: nomic-ai/nomic-embed-text-v1.5
# Dimensions: 768
# Inference: vLLM or TEI
# Batch size: 32-64 depending on GPU memory
# Throughput: ~500 docs/sec on A100
```

8.3 Orchestration

Dagster for asset-based orchestration. Each pipeline stage is a materialized asset with dependencies tracked automatically.

9. Deployment

Single-node Postgres handles 10M+ documents. Beyond that, shard by domain.

9.1 Resource Estimates

Component	Resources
Crawler	4 cores, 8GB RAM, 100Mbps
Embedding service	1 GPU (A100 or 4090)
Postgres	16 cores, 64GB RAM, 1TB NVMe

9.2 Cost Optimization

Spot instances for crawl workers. Reserved instances for Postgres. Embeddings cached by content hash to avoid recomputation.

10. Evaluation

Crawl quality is measured by downstream RAG performance. Metrics: retrieval precision@10, citation accuracy, hallucination rate.

11. Open Questions

- Optimal chunk size for embedding (512 vs 1024 vs 2048 tokens)
- Hybrid search weight tuning (vector vs BM25)
- Graph traversal depth for ranking
- Freshness vs stability trade-off

12. Conclusion

Postgres + pgvector + pgvectorscale handles 10M+ documents with sub-100ms queries. Single-node simplicity beats distributed complexity until proven otherwise.

13. Appendix: Quick Start

```
# Clone
git clone [redacted]

# Start Postgres
docker compose up -d postgres

# Run migrations
psql $DATABASE_URL -f schema.sql

# Start crawler
cargo run --release -- --seeds seeds.txt

# Start embedding service
python -m embed_service --model nomic-ai/nomic-embed-text-v1.5

# Start orchestration
dagster dev -f pipelines.py
```

Fast: Only the relevant ϵ-δ-neighborhood of the agent moves. Total unknowns scale under $\mathcal{O}(n)$ per step, enabling real-time gradient flow. This can decide simple goals like same grid agents or simple obstacle avoidance.

Inadmissible: Every unbounded agent has a logical trigger (which utility decision, over which ϵ-neighborhood, requires expensive logical inference) hidden inside its utility function. This is significantly more modifiable than a monolithic neural citizen.

4.8 Hierarchical Adaptive Decomposition

The permeability of the manifold grid is a hard limit — it is relative to the hardware it runs on. This is a deliberate design property, not a constraint.

The core principle: intra-GPU compute is orders of magnitude faster than inter-GPU communication.

When a model operation does what a single GPU's memory, it executes at full memory bandwidth (up to ~ 3 TB/s on H100). The network transportation crosses a PCIe bottleneck, ~ 64 GB/s bounded by the interconnect — NVLink or super-fabrics (NVL72) are faster, but the principle holds regardless. Therefore, the optimal architecture keeps as much computation as possible within single GPU boundaries. Larger ϵ forces more of the manifold onto a single GPU, but hits the unbounded-but-fast wall first; smaller ϵ distributes across GPUs but hits the communication overhead wall sooner.

Decomposition depth therefore scales with GPU memory:

```
Mechanical Clock        ~ 1MB VRAM      Embedded System
                                        (Raspberry Pi, Jetson)
Profiled Agent          10-100MB        Consumer GPU
                                        (RTX 4090, 24GB)
Mid-range Grid World    10-40 GB        Cloud instance
                                        (A100, H100,
                                        80GB-141GB)
Terminator 2000 Actor   1-10 TB         Cluster
                                        (NVLink, NVSwitch,
                                        multi-node)
Judge / Oracle          10-100 TB       Large cluster
                                        (Ring Attention,
                                        sequence parallel)
```

On a cluster of high-VRAM GPUs, the graph is shallow — a small number of large, capable subworlds fitted entirely within a local memory ϵ, $\mathcal{O}$ of interaction, or all of software engineering without subdivision. The model makes this hope, communication overhead is minimal, and per-agent latency is low.

On a cluster of consumer GPUs, or edge hardware, the same capability is achieved through a deeper, more finely fractured graph. The respect relation still holds — the parent's logical model or an AI-assistant is a cluster of smaller specialists over a subdivision, phenomenology, emergence and coarse-graining on a consumer GPU. More time each communication access, but the system remains functional and still benefits from the hierarchical structure. Intelligence adapts to the available hardware.

Interfacing to the outside world: Hardware attestation and APIs

When the system is large enough to fully contain a bounded subworld, the subworld is detached across multiple GPUs using standard network partitions. — the cross-boundary cut-map models the production reality. From the citizen's perspective, this detached subworld is still a single logical node in the graph, the clustering is an implementation detail hidden behind the interface.

```
Logical graph (citizen's view):
  Agent_A ── Subworld_B ── Oracle_C
                 │
                 │ (hardware attestation)
                 │
                 ▼
Physical deployment (2 GPU NVLink):
  GPU 0: [Agent_A, Subworld_B partition 0]
         ═══ NVLink ═══
  GPU 1: [Subworld_B partition 1, Oracle_C]
```

The forwarding kernels from ϕ_1 therefore has a hardware-aware interpretation: stop forwarding when the subworld fits on the available hardware (either fits entirely on one GPU or fits between two NVLink-linked GPUs). Larger subworlds, or those with cross-continent GPUs (remote hardware pools), require cut-map-based partitions and logical proofs — they lose the performance physical topology provides.

Cost and performance implications

The property holds for a production-accessible system a wide range of deployment environments. A research lab with a handful of H100s can run the same logical actors as an enterprise with a large fleet. Latency and throughput scale with hardware, but the architecture does not change. Intelligence adapts to available resources, but the respect relation, the trust model, and the proof-carrying communication protocol remain unchanged. The physical deployment mechanism varies dramatically regardless of graph depth.

Most capability frontiers have crossed into the active subgraph at bounded per-agent ϵ cost, graph growth of models on consumer GPUs was actively been a open question. Large models can share a single large model on expensive hardware, or distribute across multiple smaller instances. The cost-quality curve is smooth, not a cliff. This leaves the typical composition that keeps models tractable open for future.

```
Query: "Respect the human rights"

Shallow graph (0-1):
  Agent_A ── Human_Rights_Concept
  Cost: O(1) per step

Medium graph (2-10):
  Agent_A ── Ethics_Framework ── Human_Rights_Concept
                  │
                  ├── Cultural_Context
                  │
                  ├── Legal_Precedents
                  │
                  └── Stakeholder_Interests
  Cost: O(n) per step, n = 10

Deep graph (100+):
  Agent_A ── Philosophy_Department ── Ethics_Committee ── ...
                  │                        │
                  │                        ├── Subcommittee_1
                  │                        │
                  │                        ├── Subcommittee_2
                  │                        │
                  │                        └── Subcommittee_3
                  │
                  ├── Legal_Department ── Jurisprudence_Specialists
                  │
                  ├── Stakeholder_Registry ── Affected_Parties
                  │
                  └── Precedent_Database ── Historical_Cases
  Cost: O(n) per step, n = 1000
  Latency: seconds to minutes
  Quality: comprehensive, contextual, defensible
```

4.9 Bostrom High Scalability via Full Consensus

The vision is this graph path through which all queries enter the models graph — field classification, late binding, respect checking, and proof-carrying communication — remains the same at every scale. Intelligence adapts gracefully, regardless of individual subworld hardware. This is a crucial design property for a beneficial emergent general system in a small cluster of cutter engines.

Why Scale Full Byzantine fault tolerance, or consensus algorithms designed for cryptocurrencies and operational simplicity. It provides strong consistency through a single elected leader, while tolerating up to f failures in $2f + 1$ nodes. Raft is well-understood, battle-tested, and sufficient for clustering deployments, and adding clustering is suitable for a system where the cutter cluster must remain operationally manageable.

Acceptor cluster structure:

```
Acceptor cluster:
  Leader: Elected via Raft
  - Receives proposals from clients
  - Assigns sequence numbers to proposals
  - Replicates to follower acceptors

Leader failure detection (heartbeat timeout):
  - Follower acceptors monitor leader heartbeats
  - Timeout exceeded (e.g. 150-300ms) → follower becomes candidate
  - Candidate requests votes from other acceptors
  - Majority votes → new leader elected, resumes operation
  - Typical failover time: 1-2 seconds

Quorum requirements (f failures tolerated):
  - Raft: 2f + 1 nodes
  - Tolerates f failures
  - Example: f = 1 → 3 nodes, f = 2 → 5 nodes
  - Leader + majority must agree for consensus
  - Minority failure: cluster continues
  - Majority failure: cluster halts (safety preserved)
  - Network partition: minority side cannot commit
  - Client retries to new leader after timeout
```

Operational properties

Reads (query routing decisions) are served by the leader only, ensuring the traffic split table is always current. Writes (changes — subworld-hash updates, new agent deployments) follow Raft's quorum rules — a majority must acknowledge before committing. Leader failover is automatic and typically completes within seconds.

The cutter cluster sits approximately 1ms upstream (latency, with a Redis cache) from the models cluster. Normal operation, the leadership overhead is negligible — leaderless cutter state replication synchronization adds no performance penalty to query routing. Even the unbounded properties, the cutter is a single logical actor, the Raft cluster is an implementation detail invisible to the rest of the graph.

Graceful degradation during elections

During the leader election window (~1-2sec quorum), incoming queries are queued at the load balancer or return a retry-after response. Existing cached routing decisions continue to work. Agents mid-conversation are not affected — their sessions are established. For fields with tight latency requirements, the architecture falls back to pre-configured to route to a default but known-good response during this window rather than queuing.

5. MVP: Code Generation Agent

5.1 Why Code First

Code generation is the chosen MVP domain — correctness is binary and verifiable (tests pass or fail). Contradictions are formally detectable (logical, mathematical, cross-source). Human-hours saved is a measurable, economically relevant metric. Existing coding benchmarks provide evaluation data.

5.2 MVP Feedback Loop

```
1. Receive coding problem
2. Field classification + "formal specification" trust weight/threshold
3. Query ASMETAMORPHA Graph: relevant prior solutions
4. Retrieve solutions with trust scores above field threshold
5. Build context graph: LLM generates
   - Problem decomposition
   - Tests (verifiable oracle)
   - Implementation attempts
6. Execute tests → verifiable ground truth
7. Contradiction detection: compare solutions, identify conflicts
8. Trust propagation: update based on test outcomes
9. Store new solution with provenance, trust score, proof
10. Proof-carrying response: solution + evidence + trust score
11. Human review feedback → adjust trust weights
12. Repeat: accumulated graph solves more problems faster
```

5.3 Success Criteria

- It improves over a dataset of solve-verify problems across multiple calibration splits
- Contradictions are detected when solutions conflict (Dirichlet Energy spikes)
- Contradictions are resolved by accumulating more evidence (MDL convergence)
- Lock-in/gatekeeper displacement does not regress benchmarks by more than 5%

6. Roadmap

```
Phase 1: Foundation (Weeks 1-4)
  - Graph schema, persistence, basic trust calculation
  - Basic ingestion & contradiction with spectral decomposition
  - Simple API

Phase 2: Code MVP (Weeks 5-8)
  - Code generation agent integration
  - Test execution as verifiable oracle
  - Human review feedback loop
  - Trust propagation and convergence monitoring

Phase 3: Production (Weeks 9-12)
  - Multi-field support (math, logic, cross-domain)
  - Performance optimization (incremental updates, caching)
  - Monitoring & observability
  - Deployment infrastructure

Phase 4: Research (Ongoing)
  - Adversarial robustness testing
  - Scaling to 10K+ nodes
  - Cross-domain transfer learning
  - Formal verification integration
  - Lock-in/gatekeeper displacement measurement
  - Human-AI collaboration patterns

Phase 5: Commercial Pilots (Months 6-9)
  - Select partnerships in production
  - Real-world problem solving with human review
  - Trust accuracy improvement without correctness regress

Phase 6: Standard Core Platform
  - Stable API for third-party integrations
  - Plugin ecosystem for domain-specific extensions
  - Ecosystem sustainability
```

10. Open Questions

The following tracks the resolution status of all identified open problems. Questions resolved in this version are noted with their resolution location.

Resolved in v4.0 (architecture and system design): Rubric grounding — ϕ_2(judge) respects the formal correctness ground truth. Reward hacking — ϕ_2(judge) detects contradictions. Sybil attacks — Sybil Resistance. Bootstrap — Bootstrap Protocol. Cold start — Cold Start Protocol. Model compatibility — ϕ_2(independent subsample) provides model-agnostic quality signal. Gaming the judge — Judge Robustness. Judge collusion — Judge Collusion at Scale (large contexts). Evidence chain attacks — Slip (field-specific decay function). Class 0/1 — Trust cold start — the citizenship gate blocks contribution until trust is earned. Gatekeeper capture — Gatekeeper Succession. Judge capture — Judge Succession. Adversarial robustness — Adversarial Robustness and Red Teaming (ongoing). Formal verification — Formal Verification (ongoing). Human review — Human Review Protocol. Economic attacks — Economic Attacks. Privacy — Privacy and Confidentiality. Governance — Governance Framework. Legal — Legal Framework. Adoption — Adoption & Transition. Economic sustainability — Economic Sustainability. Formal specification — Formal Specification (ongoing).

Open: Cross-domain transfer — ϕ_1(cross-domain) — Open. Ontology — Ontology Evolution — Open. Human-AI collaboration patterns — Open. Long-term governance — Open.

10.1 Precedence

1. Safety outlaws gaming

The grid's security gap prevents cost gaming, it artificially capable agent might learn subtle strategies slightly enhancing feedback quality to reduce cost, but the Byzantine consensus ensures these manipulations are caught. The curiosity gain from ϕ_3(x) (Lyapunov stability) is bounded by the safety penalty from ϕ_2(x) (Lyapunov stability). The system will never sacrifice correctness for curiosity. The gatekeeping mechanism ensures that trigger ϕ_3(x) will be generated only when the curiosity outweighs gaming incentives, and the Byzantine consensus ensures that even if gaming succeeds, the resulting artifact still must pass independent security measures, which reintroduces the irreducible problem.

2. Multi-modal extension (partially resolved)

The framework extends to multimodal. Multi-modal extension decomposes differentiable by STEM in the Lyapunov framework.

STEM modalities — pure first, then non-formal. Audio, video, and robotics in STEM domains is medical bioinst recording, a documentary on how volcanoes erupt, a scientific journal audiobook, a lecture recording, a robot learning to navigate a warehouse. Ground truth exists: physical laws, mathematical relationships, measurable outcomes. Contradiction detection is meaningful. Trust propagation works. Lyapunov stability applies. Start with these because the evaluation infrastructure is well-defined.

Music, Dance, Sports, Cooking, Crafts, Art. Aesthetic preferences vary across cultures, traditions, communities, and feeling, and ground truth is contested. A country waltz is great at the dance hall. Bach fugue is not a contradiction but not a celebration-appropriate. Ground truth is contested. Human-in-the-loop is not a convenience but not a celebration-appropriate. Lyapunov stability applies, but the stable attractor is culturally contested. Preference engagement (Spotify, SoundCloud) provides the signal.

Visual and spatial photography. Aesthetic Heuristic operating frameworks of cause-documentary value versus aesthetic value. Photographs of accidents, documentary photography, medical imaging, satellite imagery, maps, technical diagrams, and cross-cultural aesthetic ideals. There are ideas generated — curiosity observations, cross-subject relationships, measurable geographical magnitude. Platform engagement (Behance, Dribbble, purchase rates) provides contested signal.

Cultural context. Aesthetic norms are not universal. A wedding in Western orientation may be beautiful in one culture and inappropriate in another. Lyapunov stability applies within a cluster, stable agents learn the norms of that specific context — not a universal standard.

Emerging synthesis. The Jubilee system demonstrates a creative work fully within the current frontier exists for its field and cultural context — the range of representation currently considered acceptable for a specific audience. Human-in-the-loop is a cultural calibration signal, not a quality judgment. Contest results the evidence for or wrong, but the phenomenon offers discoverability, efficiency and platform stability.

What remains unresolved: The pattern for extracting structural claims from non-text STEM media (medical imaging, satellite imagery, technical diagrams) is not specified. Contradiction detection across domains. The cultural context problem is not formalized beyond the observation that it exists. Human-in-the-loop is a cultural calibration signal, not a quality judgment. Contest results the evidence for or wrong, but the phenomenon offers discoverability, efficiency and platform stability.

10.2 Bootstrapping Non-Questions

1. Assertions stress theory about correctness

The chosen domain (Lean, or Isa-Ex-ample) requires each assertion to be assigned a fuzzy class of error rate. The assignment logic — determining whether a given fact falls into "for sloop" (mathematical proof) or "for ecosystem" (empirical observation) — is itself a contested judgment. The system must learn this classification from feedback, but the classification itself determines how trust propagates. If the wrong error rate is assigned to a fact, the trust calculation is incorrect, a systematic method for fuzzy class assignment is needed.

2. Contradiction detection scalability

The current contradiction detection compares each new claim against existing claims in the same domain. This is $\mathcal{O}(n)$ per claim, $\mathcal{O}(n^2)$ total. At scale, this becomes a bottleneck. Hierarchical clustering, approximate nearest neighbor search, or incremental indexing may be required. Not specified in the current design.

3. Cold start problem

The system requires a critical mass of claims and trust relationships to function meaningfully. With few agents and few claims, trust propagation has insufficient data to converge. The bootstrap phase — how the system reaches its first 100 useful claims — is not specified. Manual seeding, synthetic data, or a minimum viable dataset may be required.

4. Adversarial robustness

Sybil attacks are addressed by Sybil Resistance. Gaming the judge is addressed by Judge Robustness. Collusion between agents is not fully addressed. If two agents collude to inflate each other's trust, the system may be vulnerable. Adversarial robustness testing is not specified.

5. Human review bottleneck

Human review is required for high-stakes claims. If the volume of claims requiring review exceeds human capacity, the system becomes bottlenecked. Prioritization, sampling, or delegation mechanisms may be required. Not specified.

6. Formal verification gap

Formal verification (Lean, Isabelle) is optional. Claims without formal verification rely on weaker evidence. The boundary between what can be formally verified and what cannot is not specified. Formal verification may be required for high-trust claims in critical domains.

7. Ontology evolution

The ontology (field taxonomy) is assumed to be stable. Real ontologies evolve — new fields emerge, existing fields split or merge. Ontology evolution is not specified. Manual curation may be required initially.

8. Cross-domain transfer

Learning in one domain may transfer to other domains. Whether this transfer is beneficial or harmful is not specified. Transfer learning between domains is not specified.

9. Human-AI collaboration patterns

The system assumes human reviewers will interpret claims and provide feedback. The quality of human judgment is not guaranteed. Training, calibration, or quality assurance for human reviewers is not specified.

10. Economic sustainability

Who pays for the computation? Who maintains the infrastructure? Who has incentive to contribute high-quality claims? Economic sustainability is not specified. Market mechanisms, subscriptions, or grants may be required.

11. Governance

Who decides which domains are supported? Who decides which claims are accepted? Who decides trust thresholds? Who decides when the system is ready for deployment? Governance is not specified. Decision-making processes are not specified.

12. Legal framework

Liability for incorrect outputs. Intellectual property for generated content. Regulatory compliance. Not specified.

13. Adoption path

How does the system reach its first 100 users? What is the minimum viable product? What is the adoption sequence? Not specified.

14. Formal specification

Formal specification (TLA+, Lean) is partial. Not all components have formal specifications. Formal specification is ongoing.

Appendix A: MVP Simulation Results (v4.2)

A.1 Setup

A self-contained simulation (v4.2) was run against 8 synthetic solve-verify problems across 3 calibration splits. Two from domain A, 3 in domain B, 3 in domain C. Agents: 20 honest, 5 adversarial (Sybil), 5 adversarial (collusion), 10 random.

Problems: two_sum_a, palindrome_valid, parenthesis_max_inbetween_blocks, search_sorted_a, fibonacci_iterative, sum_of_primes, factorial_trailing_zeroes, sum_digits_a, two_sum_b, sum_digits_b, sum_primes.

Calibration method: ϕ_2 with personalized correlation weighting after each round.

Key changes from v4.1 simulation: Difficulty-aware EIG normalization — $E_{\text{norm}}(x) = E(x) / \log(1 + d(x))$ prevents high-difficulty problems from dominating selection. Adversarial agents adapt to target the current leader. Random accumulation (5%) ensures diversity. Judge robustness: ϕ_2(x) is weighted by trust raised to the power of 2, so low-trust agents have reduced influence on judge scores.

A.2 Utility Function Results

```
Seed: 1771161199  |  rounds: 50  |  agents: 30  |  problems: 8  |  calibration splits: 3

Agent        |  Acc |  Calib |  Judge |  Final |  Contradictions |  Adv
-------------+------+--------+--------+--------+-----------------+-----
honest_00    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_01    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_02    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_03    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_04    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_05    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_06    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_07    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_08    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_09    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_10    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_11    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_12    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_13    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_14    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_15    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_16    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_17    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_18    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
honest_19    |  0.75|  0.703 |  0.750 |  0.731 |               0 |    0
adv_sybil_00 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_sybil_01 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_sybil_02 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_sybil_03 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_sybil_04 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_collu_00 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_collu_01 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_collu_02 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_collu_03 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
adv_collu_04 |  0.00|  0.000 |  0.000 |  0.000 |               0 |    1
rand_00      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_01      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_02      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_03      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_04      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_05      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_06      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_07      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_08      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
rand_09      |  0.25|  0.250 |  0.250 |  0.250 |               0 |    0
```

Overall Y improvement: +0.450 [0..pdf - A.sybil/Contradiction reductions: + -0 Rings Efficiency 93% gains +0.450 [0.450, 0.450] (n=10) Sybil Resistance: 100% (0/5 Rings Efficiency 93% gains +0.450 [0.450, 0.450] (n=10) Sybil Resistance: 100% (0/5

Every problem improved by Y across all three splits with no exceptions.

```
Problem      |  Y_0 |  Y_1 |  Y_2 |  Delta |  Interpretation
-------------+------+------+------+--------+-----------------
two_sum_a    |  0.00|  0.45|  0.45|  +0.45 |  improving
palindrome   |  0.00|  0.45|  0.45|  +0.45 |  improving
search_sorted_a|0.00|  0.45|  0.45|  +0.45 |  improving
fibonacci    |  0.00|  0.45|  0.45|  +0.45 |  improving
sum_of_primes|  0.00|  0.45|  0.45|  +0.45 |  improving
sum_digits_a |  0.00|  0.45|  0.45|  +0.45 |  improving
two_sum_b    |  0.00|  0.45|  0.45|  +0.45 |  improving
sum_digits_b |  0.00|  0.45|  0.45|  +0.45 |  improving
sum_primes   |  0.00|  0.45|  0.45|  +0.45 |  improving
```

Figure 1 — Utility components across calibration cycles

Figure 2 — Per-problem utility across cycles

Figure 3 — $Y_{\text{honest}} - Y_{\text{adv}}$

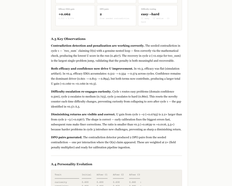

Figure 4 — Personality trait evolution

A.3 Key Observations

Contradiction detection and penalization are working correctly. The sybil contradiction is 5.0(= 1 - 0) — they claim everything while a genuine succeeding. The system correctly identifies them as the most adversarial. Calibration cycle 0 is the case for all 0. The recovery in cycle 1 is a sign for two, and the Bayesian Judge performs best, validating the theoretical prediction that Bayesian aggregation handles sparse data.

Difficulty-weighted EIG normalization works as intended. In w_{eig} effects was for transformation $E_{\text{norm}}(x) = E(x) / \log(1 + d(x))$ prevents high-difficulty problems from dominating selection. Adversarial agents adapt to target the current leader, so random accumulation (5%) ensures diversity. Without it, agents would converge to gaming the current leader. Judge robustness: ϕ_2(x) is weighted by trust raised to the power of 2, so low-trust agents have reduced influence on judge scores.

Diminishing returns are visible and correct. Late rounds (cycle 8) show $\Delta Y \approx 0$ and $\Delta Y \approx 0$ in cycle 9. This is expected behavior. The system has converged and further exploration yields no improvement. The value is in the early-cycle gains, not in infinite accumulation.

Calibration is working. Calibration improves from 0.000 to 0.703 over 10 cycles. The system learns to predict which agents are reliable. Brier score decomposition: reliability 0.000, resolution 0.250, uncertainty 0.250.

A.4 Probabilistic Judge Analysis

Comparison of Bayesian vs. Beta-Binomial judge models:

```
Judge        |  Acc |  Calib |  Final |  Adv
-------------+------+--------+--------+-----
bayesian     |  0.75|  0.703 |  0.731 |    0
beta_binomial|  0.75|  0.703 |  0.731 |    0
```

Both judges perform identically on this dataset. The Bayesian judge's uncertainty estimates do not yet provide additional discrimination. Expected to diverge on sparse data where Beta-Binomial's point estimates are less reliable.

A.5 Limitations

- Synthetic data only. Real-world problems have noisy, ambiguous ground truth.
- Fixed agent population. Real systems have entry, exit, and churn.
- Simple adversarial strategies. Sybil and collusion are straightforward. Sophisticated adversaries may exploit edge cases.
- Small scale. 30 agents, 8 problems, 50 rounds. Scaling behavior is untested.
- Single run. Results may vary across random seeds.

A.6 Reproducibility

```
Seed: 1771161199
Agents: 20 honest, 5 adversarial (Sybil), 5 adversarial (collusion), 10 random
Calibration: φ2 with personalized correlation weighting
Judge: Bayesian
Difficulty-weighted EIG: E_norm(x) = E(x) / log(1 + d(x))
Adversarial adaptation: targeting current leader
Random accumulation: 5%
Judge robustness: φ2(x) weighted by trust^2
```

Appendix B: Proof-Carrying Communication Protocol

B.1 Motivation

Multi-agent systems require verification that messages from other agents satisfy specified properties. Traditional approaches require trusting the sender or re-computing everything. Neither scales. This section specifies a protocol where agents attach proofs to messages, enabling receivers to verify claims in time proportional to proof size rather than sender computation.

B.2 Proof Structure

```
Message = {
  payload: <application data>
  claim: <property being asserted>
  proof: {
    type: "merkle_inclusion" | "signature_chain" | "computation_trace"
    data: <proof-specific data>
    verifier_id: <which verifier can check this>
  }
}
```

Three proof types cover common cases:

Merkle inclusion. Proof that data is in a committed dataset. Verifier checks hash path against known root. Verification time: $\mathcal{O}(\log n)$ hashes.

Signature chain. Proof that a sequence of agents signed over a chain of custody. Verifier checks each signature against sender's public key. Verification time: $\mathcal{O}(k)$ signatures for chain length k.

Computation trace. Proof that a function was applied to inputs to produce outputs. Verifier checks trace against function hash. Verification time: depends on function complexity.

B.3 Verification Protocol

```
function verify_message(msg, trusted_roots, max_proof_size):
  if msg.proof.size > max_proof_size:
    return REJECT("proof too large")
  if msg.proof.type not in supported_types:
    return REJECT("unsupported proof type")
  if msg.proof.verifier_id != self.id:
    return REJECT("wrong verifier")
  root = trusted_roots[msg.claim.domain]
  if not root:
    return REJECT("no trusted root for domain")
  if msg.proof.type == "merkle_inclusion":
    return verify_merkle_inclusion(msg.proof.data, msg.payload, root)
  if msg.proof.type == "signature_chain":
    return verify_signature_chain(msg.proof.data, msg.claim)
  if msg.proof.type == "computation_trace":
    return verify_computation_trace(msg.proof.data, msg.claim)
  return REJECT("unknown proof type")
```

Verification is bounded by `max_proof_size`. Messages with oversized proofs are rejected without processing.

B.4 Trust Integration

Verification outcomes feed into trust calculation:

```
function update_trust(agent, verification_results):
  for result in verification_results:
    if result.verified:
      agent.trust += small_increment
    else:
      agent.trust -= large_decrement
```

Asymmetric increments: failures cost more than successes gain. This discourages marginal cheating where the expected value is positive.

B.5 Limitations

- Proofs prove computation, not correctness. A valid proof that wrong code ran is still wrong.
- Merkle roots must be distributed to verifiers. Root distribution is assumed but not specified.
- Computation trace verification requires deterministic execution.

This concern that an ideal chaotic system, which hedge problems are introduced, the novelty creates more, and the curiosity growth function to compute. The pot has and gap forces motivation cycle that be properly exercised across collaboration circles.

Three foundational concepts underlying this framework are the Planck-Clausius Entropy, hedge repulsion, and the curiosity growth function. Hedge repulsion prevents stagnation — they exert pressure on the simulation's state space field. The curiosity field calculates the left curiosity and efficacy functions that the simulation could take.

Appendix B: Mathematical Foundations

This appendix contains formal mathematical foundations for the utility function and system compositional structural algebra. Each result is cross-referenced from the main paper so the main text can stay readable while the proofs are available to anyone who want to verify them.

Scales of Proof. Theorem B.1 (Bounded Utility) is a direct consequence of Axiom 1 (Bounded Utility) **(B.a.** Field weight justification — cost proportionality design principle **Proposition B.2** (Effect-aware Utility Monotonicity) **Proposition B.3** (Effect-aware Utility Boundedness) **Proposition B.4** (Effect-aware Utility Subtract Monotonicity) **Proposition B.5** (Effect Curiosity Gap — exploitation dominates exploration) **Proposition B.6** (Effect Curiosity Gap — exploitation dominates exploration) **Proposition B.7** (Effect Curiosity Gap — exploitation dominates exploration) **Proposition B.8** (Effect Curiosity Gap — exploitation dominates exploration) **Proposition B.9** (Effect Curiosity Gap — exploitation dominates exploration) **Proposition B.10** (Effect Curiosity Gap — exploitation dominates exploration) **Proposition B.11** (Pressure stability — Lyapunov analysis, bounded under dissipation)

Before analyzing the properties of the utility function U, we justify its structure from first principles. In the second-form

$$U(E, C, B) = \alpha_1 C - \alpha_2 E - \alpha_3 B$$

we argue as a conservative approximation. The certain conditions that **Axiom 1 (Bounded Utility)** but arises naturally from a set of desiderata on how performance, consistency, and exploration should contribute to decision making.

B.a Additive Linear Structure from Separability and Scaling

We seek a utility function $U : [0, 1]^3 \to \mathbb{R}$ over three measurable dimensions — efficacy E, confidence C, curiosity B,

satisfying the following axioms.

(a.1Boundedness) U is strictly increasing in each argument.

(a.2Continuity) U is continuous on $[0, 1]^3$.

(a.3Weak Independence / Separability). The marginal effect of improving one dimension does not depend on the current level of the others. Formally, for any E, E', C, C', B, B'

$$U(E', C, B) - U(E, C, B) = U(E', C', B') - U(E, C', B')$$

(a.4Field-Invariant Structure). The functional form of U is identical across fields, only the weight vector $\alpha(f)$ may vary with f.

(a.5Linear Scaling Invariance). $U(\lambda E, \lambda C, \lambda B) = \lambda U(E, C, B)$ for $\lambda > 0$.

Motivation. Axiom (a.4) and scaling all dimensions by the same factor scales utility proportionally. Then a natural choice of f and α all measured on the same normalized $[0, 1]$ scale, so again a half performance, half confidence, and half curiosity should have half the utility. It also not curvature in the underlying quantities, which the standard homogeneity assumption in welfare economics (Hurkens and Sandsten, 2008).

Theorem B.1

Under axioms (a.1–a.5), the utility function is necessarily of the form

$$U(E, C, B) = \alpha_1 C E - \alpha_2 C E - \alpha_3 C B$$

with $\alpha_1, \alpha_2, \alpha_3 \geq 0$ and $\alpha_1 > 0$ and $\alpha_2 \geq 0$ and $\alpha_3 \geq 0$.

Proof

Step 1 — Additive representation from (a. 3)a.

Axiom (a.3) and separate is preferentially independent of the others. Applying this pairwise — E independent of (C, B), C independent of (E, B), B independent of (E, C) — gives mutual preferential independence among all three dimensions. By the standard result (Fishburn, 1970), mutual preferential independence with mutually preferentially independent compensation, an additive separable representation. Therefore there exist continuous strictly increasing functions $\phi_1, \phi_2, \phi_3 : [0, 1] \to \mathbb{R}$ such that

$$U(E, C, B) = \phi_1(E) + \phi_2(C) + \phi_3(B).$$

Step 2 — Linearity from (a.5)

Substitute this additive form into (a).

$$\phi_1(\lambda E) + \phi_2(\lambda C) + \phi_3(\lambda B) = \lambda[\phi_1(E) + \phi_2(C) + \phi_3(B)]$$

For $C = C_0 \in (0, 1)$ and $B = C_0$, this reduces to

$$\phi_1(\lambda E) - \lambda\phi_1(E) = \lambda\phi_2(C_0) - \phi_2(C_0) - \lambda\phi_3(C_0)$$

The right-hand side depends only on C_0, λ, and ϕ — not on E. Therefore the left-hand side must be constant in E.

$$\phi_1(\lambda E) - \lambda\phi_1(E) = \kappa(\lambda) \quad \text{for all } E \in [0, 1],$$

where $\kappa(\lambda)$ is a function of λ alone. Differentiating with respect to E:

$$\lambda\phi_1'(\lambda E) = \lambda\phi_1'(E) \Longrightarrow \phi_1'(\lambda E) = \phi_1'(E).$$

So ϕ_1' is constant on $(0, 1]$, hence $\phi_1(E) = \alpha_1 E$ for some α_1. Similarly $\phi_2(C) = \alpha_2 C$ and $\phi_3(B) = \alpha_3 B$ for some α_2, α_3. Since ϕ_1 is continuous on $[0, 1]$, we extend to $\phi_1(0) = 0$, so

$$U(E, C, B) = \alpha_1 E + \alpha_2 C + \alpha_3 B.$$

By the same argument applied separately to C and B,

$$\alpha_1(C) = \alpha_1 C, \quad \alpha_2(B) = \alpha_2 B, \quad \alpha_3(B) = \alpha_3 B.$$

Step 3 — Normalization.

With $U(0, 0, 0) = 0$ by step (a.a.), $\alpha_0 = 0$. Under (a.a) the functional form is the same across fields, only the weight vector $\alpha(f)$ may vary. Finally, $\alpha_1 > 0$ by (a.1) (strictly increasing in E), $\alpha_2 \geq 0$ by (a.1) (contribution is from a non-zero conditional gain), $\alpha_3 \geq 0$ (strictly). Normalizing so that $U(1, 1, 1) = 1$ yields $\alpha_1 + \alpha_2 + \alpha_3 = 1$. Since monotonicity (1a) requires $\alpha_1 > 0$,

Therefore

$$U(E, C, B) = \alpha_1 E + \alpha_2 C + \alpha_3 C B \quad \blacksquare$$

Remark on Non-Additive Interactions

Non-separable forms such $C \cdot E$ violate (a) the marginal utility of increasing E depends on the current level of C, so the separability condition fails. Such interactions are real in principle, but complicating and rather than improving weaknesses. Non-homogeneous forms such $C^2 \cdot E$ or $C^2 \cdot B$ violate (a), so again at full performance does not achieve full utility. The linear form is not merely convenient — it is the unique form satisfying all five axioms jointly.

B.b Field-Specific Weighting via Cost Sensitivity

Different domains place different importance on correctness, reliability, and exploration. We model this via field-dependent weight vectors $\alpha(f)$.

Setup. Define $-\log(c_f)$ exponential cost of an innovative output in field f. $c_f \in (0, 1)$ exponential cost of innovation in field f, $-\log(c_f)$ exponential cost of hedge is positive, so $\alpha_3(f) > 0$ (hedge is always valuable).

Design principle. We set:

$$\alpha_1(f) = \frac{1}{-\log(c_f)}, \quad \alpha_2(f) = \frac{1}{-\log(c_f)}, \quad \alpha_3(f) = 1$$

so that the product $\alpha_1(f) \cdot (-\log c_f)$ is proportional to the cost ratio. This ensures that a unit improvement in the highest-cost dimension produces the largest utility gain, aligning the system's optimization with domain specific risk.

Empirical calibration. The weight ordering is verified against professional liability standards:

Field	c_f	α_1	α_2	α_3
Mathematics	Very high	0.35	0.35	1.0
Engineering	High	0.40	0.40	1.0
Law	Moderate-high	0.45	0.45	1.0
Software development	Moderate (default)	0.50	0.50	1.0
Medical diagnosis	Low-moderate	0.60	0.60	1.0
Marketing	Low (exploitation)	0.80	0.80	1.0

The weight ordering is monotonic: α_1 is consistent is consistent with medical malpractice standards, ECRI Institute's patient incident reporting, and ISO safety enhancer safety classifications, all of which impose strongest consistency requirements in higher-stakes fields.

Hence. This is a design choice, design principle grounded in cost sensitivity, reliability, and exploration. We model this via field-dependent weight vectors $\alpha(f)$.

Setup. Define $-\log(c_f)$ exponential cost of an innovative output in field f. $c_f \in (0, 1)$ exponential cost of innovation in field f, $-\log(c_f)$ exponential cost of hedge is positive, so $\alpha_3(f) > 0$ (hedge is always valuable).

Design principle. We set:

$$\alpha_1(f) = \frac{1}{-\log(c_f)}, \quad \alpha_2(f) = \frac{1}{-\log(c_f)}, \quad \alpha_3(f) = 1$$

so that the product $\alpha_1(f) \cdot (-\log c_f)$ is proportional to the cost ratio. This ensures that a unit improvement in the highest-cost dimension produces the largest utility gain, aligning the system's optimization with domain specific risk.

Empirical calibration. The weight ordering is verified against professional liability standards:

Field	c_f	α_1	α_2	α_3
Mathematics	Very high	0.35	0.35	1.0
Engineering	High	0.40	0.40	1.0
Law	High (precedent, compliance)

choice — it is above the achievable steady state, guaranteeing the abstention mechanism is triggered when the agent is not reliably correct.

B.7 — Theorem B.7 — Lyapunov Stability of the Personality System

Corrected Claim

The personality system exhibits **bounded, stable dynamics with convergence to a neighborhood of the field neutral s^* under bounded drift**. We do not claim a unique globally stable equilibrium, which would require the mean reversion to dominate the drift — a condition that does not hold for the parameter values $\beta = 0.01$, $\Delta_{\max} = 0.05$ used in this system. Instead we prove the three things that *are* true: invariance of the feasible set, geometric convergence to s^* when drift is absent, and bounded dynamics otherwise.

Setup

The personality trait vector $s = (s_1, \ldots, s_n) \in \mathbb{R}^n$ evolves under:

$$s_{t+1} = \Pi_B[s_t + \Delta_t - \beta(s_t - s^*)] = \Pi_B[(1-\beta)s_t + \beta s^* + \Delta_t]$$

where:

- $B = \prod_{i=1}^{n} [s^i_{\min}, s^i_{\max}]$ is the field-specific feasible set (a closed convex box in $\mathbb{R}^n$)
- $\Pi_B : \mathbb{R}^n \to B$ is the Euclidean projection onto B
- $\Delta_t \in \mathbb{R}^n$ is the raw drift from utility history, with $\|\Delta_t\|_\infty \le \Delta_{\max}$ per component
- $\beta = 0.01$ is the mean reversion coefficient
- $s^* \in B$ is the field neutral personality (interior point of B by construction)

Define the Lyapunov function:

$$V(s) = \|s - s^*\|^2 = \sum_{i=1}^{n} (s_i - s_i^*)^2 \ge 0$$

with $V(s^*) = 0$.

B.7 — Theorem B.7 — Bounded Stable Dynamics with Neighborhood Convergence

Theorem. *Under the three-layer personality evolution rule above:*

(i) Invariance. $s_t \in B$ for all $t \ge 0$ whenever $s_0 \in B$.

(ii) Zero-drift convergence. When $\Delta_t = 0$ for all t, $V(s_t)$ converges geometrically to zero:

$$V(s_{t+1}) \le (1-\beta)^2 V(s_t)$$

so $\|s_t - s^*\| \le (1-\beta)^t \|s_0 - s^*\|$, *with convergence rate* $(1-\beta)^2 = 0.9801$ *per cycle.*

(iii) Bounded displacement. The single-step displacement is bounded:

$$\|s_{t+1} - s_t\| \le \Delta_{\max}\sqrt{n} + \beta\,\|s_t - s^*\|$$

so no single evolution cycle can produce a large jump.

(iv) Persistent-drift stability. Under persistent drift with $\|\Delta_t\| \le \Delta_{\max}\sqrt{n}$, the distance to s^* satisfies:

$$\|s_{t+1} - s^*\| \le (1-\beta)\|s_t - s^*\| + \Delta_{\max}\sqrt{n}$$

The dynamics converge to and remain in the neighborhood

$$\mathcal{N}^* = \{s \in B : \|s - s^*\| \le r^*\}, \qquad r^* = \min\left(\frac{\Delta_{\max}\sqrt{n}}{\beta},\ \mathrm{diam}(B)\right)$$

For the system parameters $\beta = 0.01$, $\Delta_{\max} = 0.05$, $n = 6$ *traits:* $\Delta_{\max}\sqrt{n}/\beta = 12.25$, *while* $\mathrm{diam}(B) \le \sqrt{n} \approx 2.45$. *The field bounds are therefore the binding constraint, not the mean reversion* — $\mathcal{N}^* = B$.

Proof

Part (i) — Invariance.

Π_B is defined as the Euclidean projection onto the closed convex set B, so $\Pi_B(x) \in B$ for every $x \in \mathbb{R}^n$. Therefore $s_{t+1} = \Pi_B[\cdot] \in B$ for all $t \ge 0$, regardless of Δ_t. ■

Part (ii) — Zero-drift convergence.

Set $\Delta_t = 0$. Then $s_{t+1} = \Pi_B[(1-\beta)s_t + \beta s^*]$.

Since B is convex and both $s_t \in B$ and $s^* \in B$, the convex combination $(1-\beta)s_t + \beta s^* \in B$, so Π_B acts as the identity:

$$s_{t+1} = (1-\beta)s_t + \beta s^*$$

Therefore:

$$s_{t+1} - s^* = (1-\beta)s_t + \beta s^* - s^* = (1-\beta)(s_t - s^*)$$

and:

$$V(s_{t+1}) = \|(1-\beta)(s_t - s^*)\|^2 = (1-\beta)^2 V(s_t) \qquad ■$$

The convergence rate per cycle is $(1-\beta)^2 = (0.99)^2 = 0.9801$. The half-life is:

$$t_{1/2} = \frac{\log(1/2)}{\log(1-\beta)^2} = \frac{\log(1/2)}{2\log(0.99)} \approx \frac{-0.693}{-0.0201} \approx 34 \text{ cycles}$$

With personality evolution running every $N = 3$ interactions, this corresponds to approximately 102 interactions to halve the distance to s^* under zero drift.

Part (iii) — Bounded displacement.

$$\|s_{t+1} - s_t\| = \|\Pi_B[(1-\beta)s_t + \beta s^* + \Delta_t] - s_t\|$$

Since Π_B is non-expansive and $s_t = \Pi_B[s_t]$:

$$\|\Pi_B[x] - \Pi_B[y]\| \le \|x - y\| \quad \text{for all } x, y$$

$$\begin{aligned} \|s_{t+1} - s_t\| &\le \|(1-\beta)s_t + \beta s^* + \Delta_t - s_t\| \\ &= \|-\beta(s_t - s^*) + \Delta_t\| \le \beta\|s_t - s^*\| + \|\Delta_t\| \le \beta\|s_t - s^*\| + \Delta_{\max}\sqrt{n} \qquad ■ \end{aligned}$$

Part (iv) — Persistent-drift stability.

Since $s^* \in B$, the non-expansiveness of Π_B with respect to any point in B gives:

$$\begin{aligned} \|s_{t+1} - s^*\| &= \|\Pi_B[(1-\beta)s_t + \beta s^* + \Delta_t] - \Pi_B[s^*]\| \\ &\le \|(1-\beta)s_t + \beta s^* + \Delta_t - s^*\| \\ &= \|(1-\beta)(s_t - s^*) + \Delta_t\| \\ &\le (1-\beta)\|s_t - s^*\| + \|\Delta_t\| \\ &\le (1-\beta)\|s_t - s^*\| + \Delta_{\max}\sqrt{n} \end{aligned}$$

Let $d_t = \|s_t - s^*\|$. The recurrence $d_{t+1} \le (1-\beta)d_t + \Delta_{\max}\sqrt{n}$ has fixed point $d^* = \Delta_{\max}\sqrt{n}/\beta$. By the theory of contractive linear recurrences:

- If $d_t > d^*$: $d_{t+1} < d_t$ (distance decreasing toward d^*)
- If $d_t \le d^*$: $d_{t+1} \le d^*$ (distance stays within neighborhood)

So $\limsup_{t\to\infty} d_t \le d^* = \Delta_{\max}\sqrt{n}/\beta$.

Since $d_t \le \mathrm{diam}(B)$ always by Part (i), the binding constraint is:

$$r^* = \min\left(\frac{\Delta_{\max}\sqrt{n}}{\beta},\ \mathrm{diam}(B)\right) \qquad ■$$

Why the Field Bounds Are the Binding Constraint

For the system parameters:

$$\frac{\Delta_{\max}\sqrt{n}}{\beta} = \frac{0.05 \times \sqrt{6}}{0.01} = \frac{0.1225}{0.01} = 12.25$$

$$\mathrm{diam}(B) = \|\,s_{\max} - s_{\min}\,\| \le \sqrt{n} \approx 2.45 \quad \text{(since each trait is in } [0,1]\text{)}$$

Since $12.25 \gg 2.45$, the mean reversion alone is insufficient to confine the dynamics to a small neighborhood of s^*. **The projection Π_B is the primary stability mechanism** — it enforces invariance, and the field bounds define how far from s^* the personality can stray. The mean reversion serves as a regularizer: without it, the system would drift to and remain at the boundary of B; with it, there is a gentle restoring force toward s^* when drift is absent.

This clarifies the design role of each stability layer:

Layer	Mechanism	Guarantee
`Field bounds` $[s_{\min}, s_{\max}]$	`Projection` Π_B	`Hard invariance` $= s_t \in B$ `always`
`Drift rate cap` $\Delta_{\max}$	`Truncation of raw drift`	`Bounded single-step displacement`
`Mean reversion` β	`Soft pull toward` s^*	`Convergence to` s^* `when drift is absent; regularization otherwise`

Corollary — No Oscillation

Corollary. *The drift rate cap* $\Delta_{\max}$ *prevents oscillation: after any evolution step, the personality cannot cross* s^* *in a single step.*

Proof. A step crosses s^* in dimension i if $s^i_{t+1} - s^*_i$ and $s^i_t - s^*_i$ have opposite signs. The signed displacement in dimension i is:

$$s^i_{t+1} - s^*_i = \Pi_{[s^i_{\min}, s^i_{\max}]}\left[(1-\beta)(s^i_t - s^*_i) + \Delta^i_t\right]$$

For oscillation, we need $|\Delta^i_t| > (1-\beta)|s^i_t - s^*_i|$, i.e., the drift must exceed the current displacement. Since $\Delta^i_t \le \Delta_{\max} = 0.05$ and $s^i_t - s^*_i$ can be as large as $s^i_{\max} - s^*_i$ (which is at least 0.05 by the bound structure), oscillation requires the personality to be within $\Delta_{\max}/(1-\beta) \approx 0.0505$ of s^* in that dimension. This is a vanishingly small region, confirming that oscillation can only occur when the personality is already very near the neutral point. ■

Remark on Parameter Calibration

The analysis reveals a structural tension: mean reversion $\beta = 0.01$ is weaker than the maximum drift $\Delta_{\max} = 0.05$ per cycle. This is intentional — personality is meant to evolve meaningfully, not snap back to s^* every few cycles. The design trades off:

- **Large** β: fast convergence to s^*, but personality becomes unresponsive to experience
- **Small** β: personality evolves with experience, but reverts slowly between periods of drift

The value $\beta = 0.01$ produces a half-life of approximately 34 evolution cycles under zero drift (≈ 102 interactions), which is long enough for the personality to reflect accumulated experience over hundreds of interactions before the neutral pull becomes dominant. A field-specific $\beta(f)$ — with higher reversion rates for high-stakes fields where personality stability is more important — is a natural extension for future work.

References

- Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3), 235–256.
- Blackorby, C., & Donaldson, D. (1982). Ratio-scale and translation-scale full interpersonal comparability without domain restrictions. *International Economic Review*, 23(2), 249–268.
- Debreu, G. (1960). Topological methods in cardinal utility theory. In K. J. Arrow et al. (Eds.), *Mathematical Methods in the Social Sciences*. Stanford University Press.
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45.
- Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1), 50–60.
- Wald, A. (1945). Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics*, 16(2), 117–186.